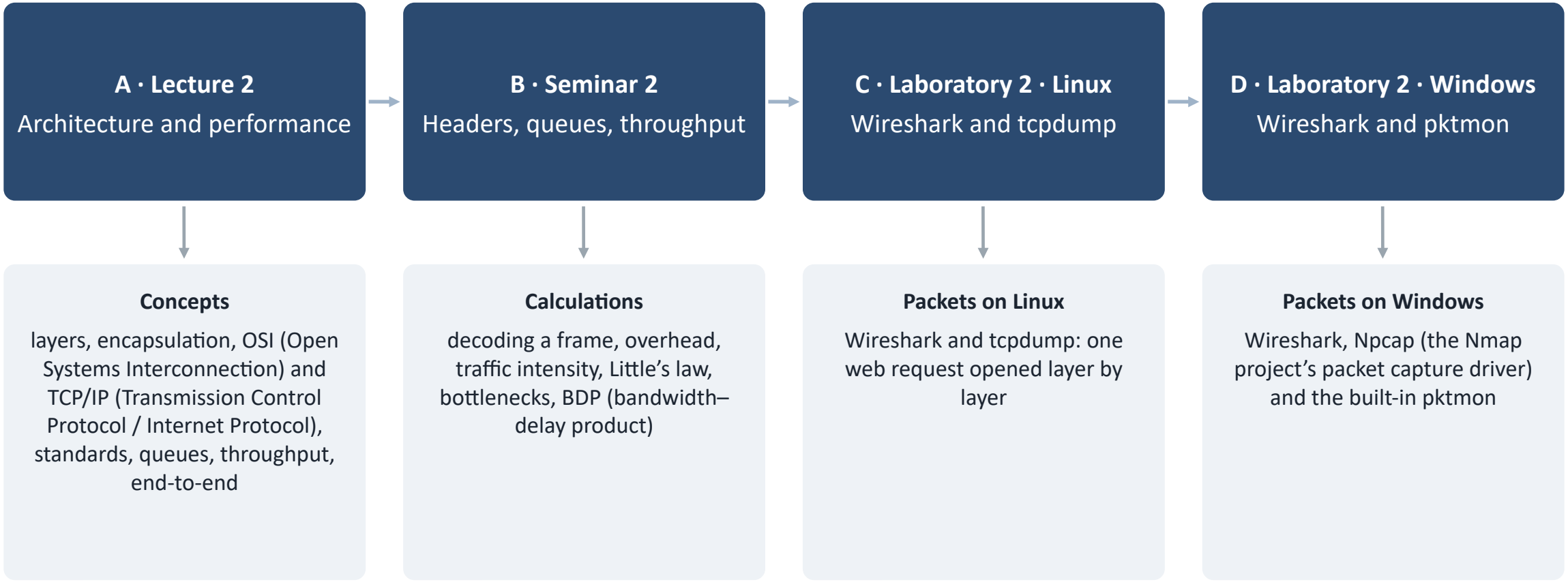
COMPUTER NETWORKS · SEMESTER 1 · WEEK 2

Architecture and performance in schemes

Layers, reference models and the numbers behind delay — lecture, seminar and laboratories

Assoc. Prof. Nicolaie Popescu-Bodorin, PhD · 2026
+40722882471 · bodorin(on)live.com · https://lmrec.org/bodorin/

How this week is organised



Key idea Read the deck left to right: the laboratories open, in real packets, the headers the lecture draws.

Part A · Lecture 2: architecture and performance

A · Lecture 2
Architecture and performance

→

B · Seminar 2
Headers, queues, throughput

→

C · Laboratory 2 · Linux
Wireshark and tcpdump

→

D · Laboratory 2 · Windows
Wireshark and pktmon

In this part

1 · Two questions and a puzzle

2 · Why layers: service, interface, protocol

3 · Encapsulation and demultiplexing

4 · Reference models: OSI, TCP/IP, the narrow waist

5 · Who makes the rules: standards and RFCs

6 · Queueing delay and loss

7 · Throughput and the bandwidth–delay product

8 · The end-to-end argument

9 · How ping and traceroute measure delay

Summary · review · exercises

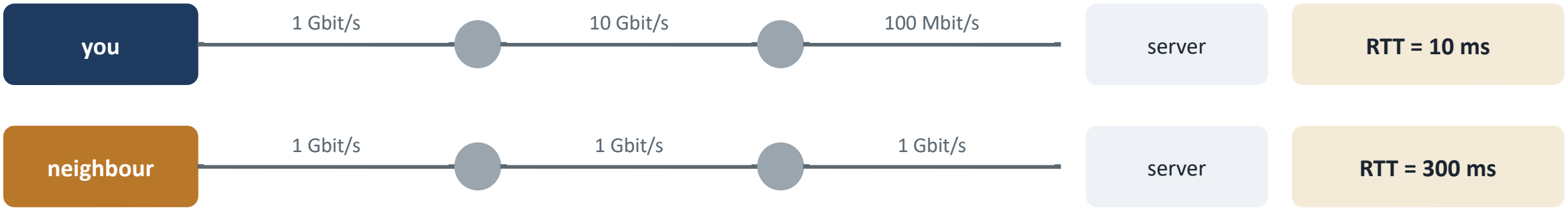
TCP (Transmission Control Protocol) · OSI (Open Systems Interconnection) · RFC (Request for Comments) · IP (Internet Protocol)

1 · Two questions, and a puzzle to keep in mind

How is the work divided?
why layers, which layers, what passes between them → sections 2–5

How well does it work?
delay, loss, throughput — and why load near 100 % breaks a network → sections 6–9

Question to start: who finishes first?



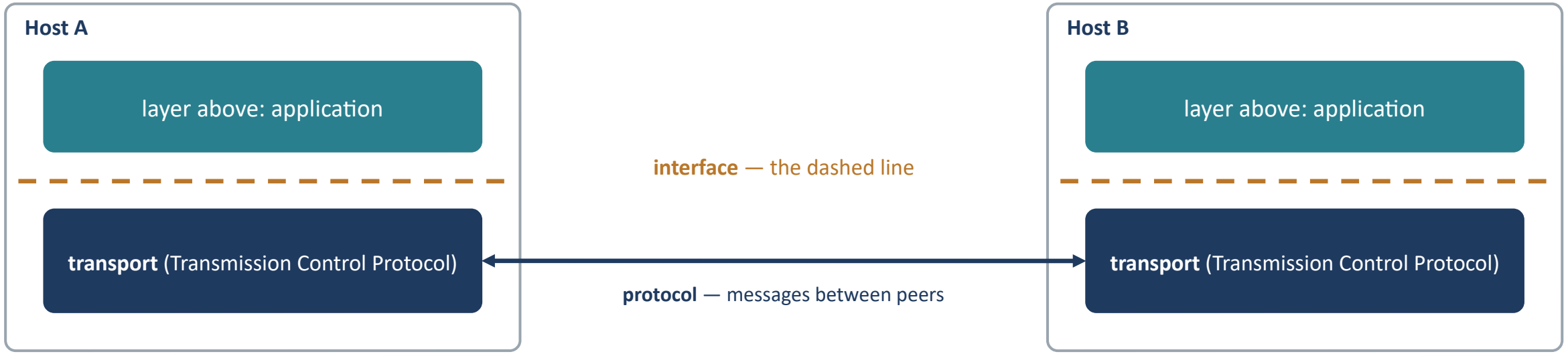
a **1 MB** download — who finishes first? ?

a **10 GB** download — who finishes first? ?

Key idea Keep your guesses: the answer comes in section 7, after delay and throughput.

RTT (round-trip time)

2 · Service, interface, protocol — three words to keep apart



1 Service

What a layer offers to the layer above.

e.g. reliable, ordered delivery of a byte stream between two processes

2 Interface

How the layer above asks for it: operations and parameters.

e.g. the socket calls `connect`, `send`, `recv`, `close`

3 Protocol

How the layer implements it, by messages with its peer.

e.g. TCP segments: sequence numbers, ACKs (acknowledgements), retransmissions

Key idea Service = what is offered, interface = how you ask for it, protocol = how it is done with the peer.

2 · A fixed interface lets the protocol underneath change

web browser — written once, unchanged

socket interface: connect · send · recv · close — fixed for decades

TCP (Transmission Control Protocol) underneath — its congestion-control algorithm replaced over the years:

Reno
late 1980s – 1990s

→

CUBIC
Linux default since the 2000s

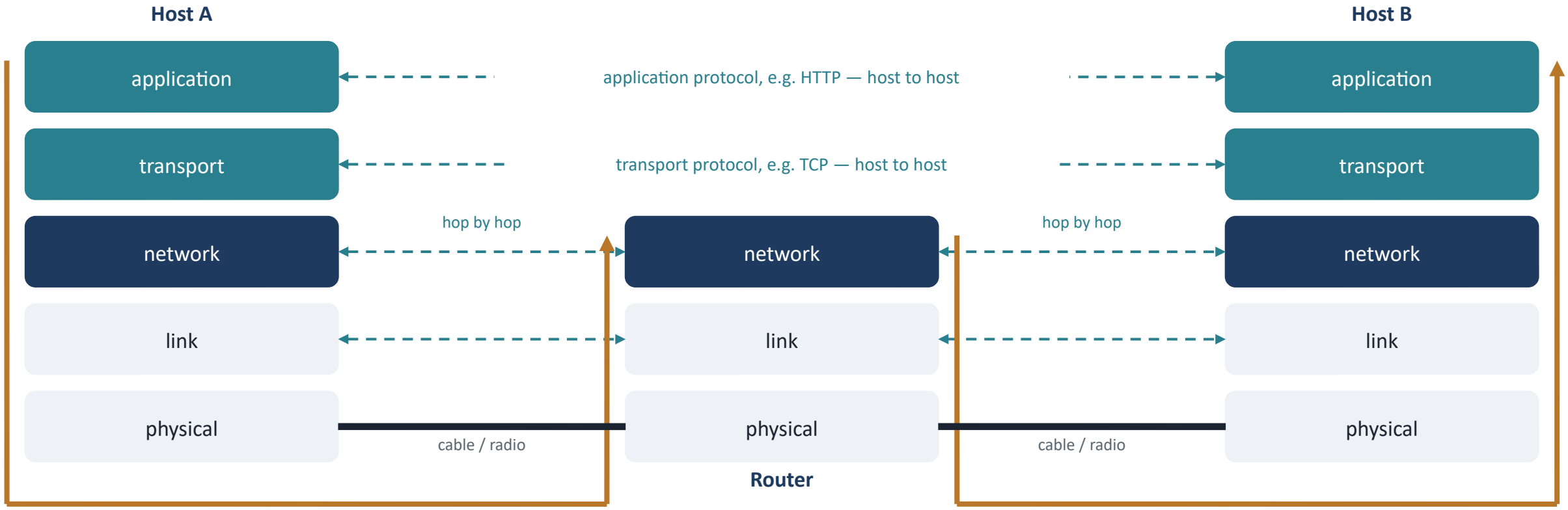
→

BBR
2016 → (Bottleneck Bandwidth and Round-trip propagation time)

1 The **service** (reliable byte stream) and the **interface** stayed; only the **protocol** inside changed.

Key idea Separating service and interface from protocol is what makes change possible without breaking applications.

2.1 · Peer communication: logical across, real down–across–up



1 **Dashed teal**: logical conversation between peers.

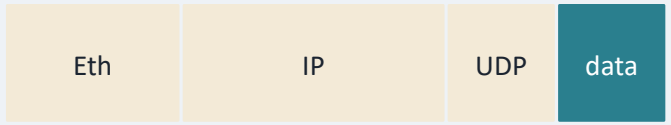
2 **Solid amber**: the real path of the data.

Key idea Routers take no part in application and transport protocols: those run only between the two hosts.

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol)

2.2 · The price of layering

1 Overhead



a 20-byte message in an 86-byte frame: **23 %** useful

Every layer adds a header — small messages pay most.

2 Duplicated functions

transport: checksum ✓

link: CRC (cyclic redundancy check) ✓

sometimes both needed, sometimes not (§ 8)

Error detection in the link layer **and** in transport.

3 Hidden information

TCP: “a packet was lost”

layer boundary

link knows: congestion?
or radio interference?

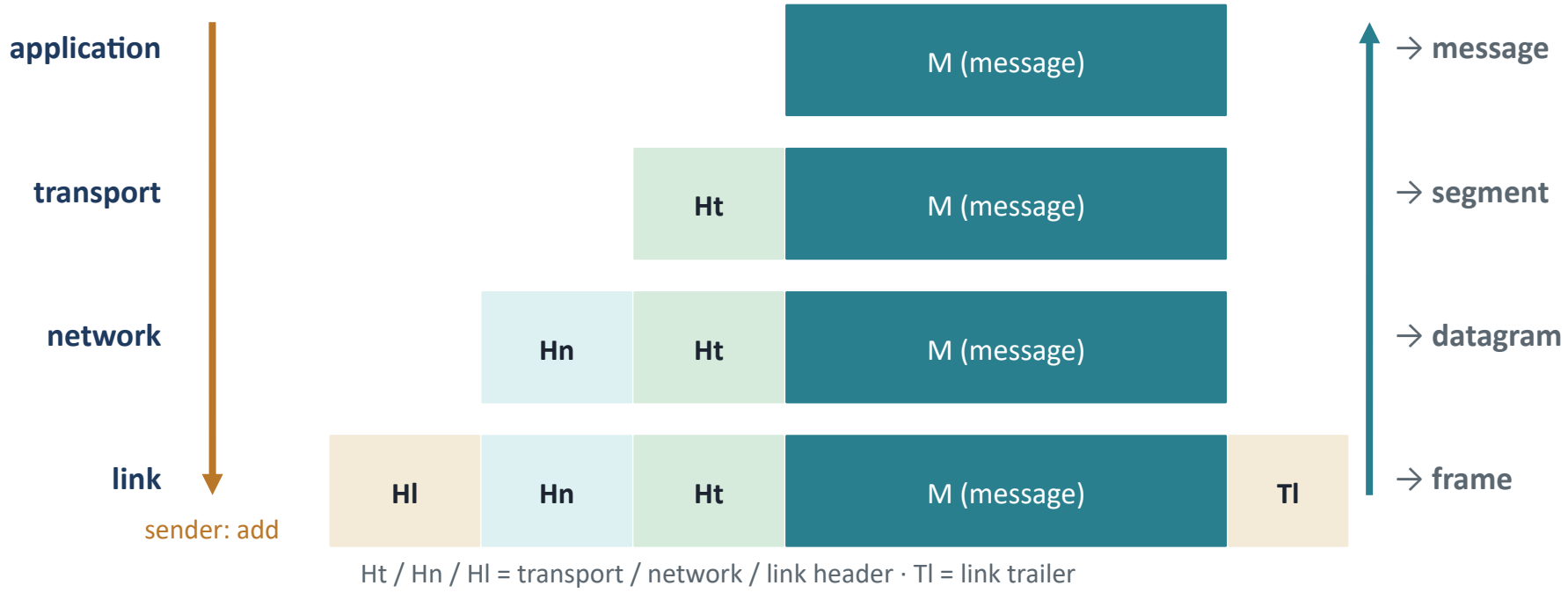
TCP sees a loss but cannot see **why**.

Cross-layer design — a deliberate exception: e.g. a video application on a phone adapting to the radio conditions it is told about.

Key idea Layers cost header bytes, repeated work and hidden information — a price usually worth paying.

TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · IP (Internet Protocol)

3 · Encapsulation: every layer wraps what it receives

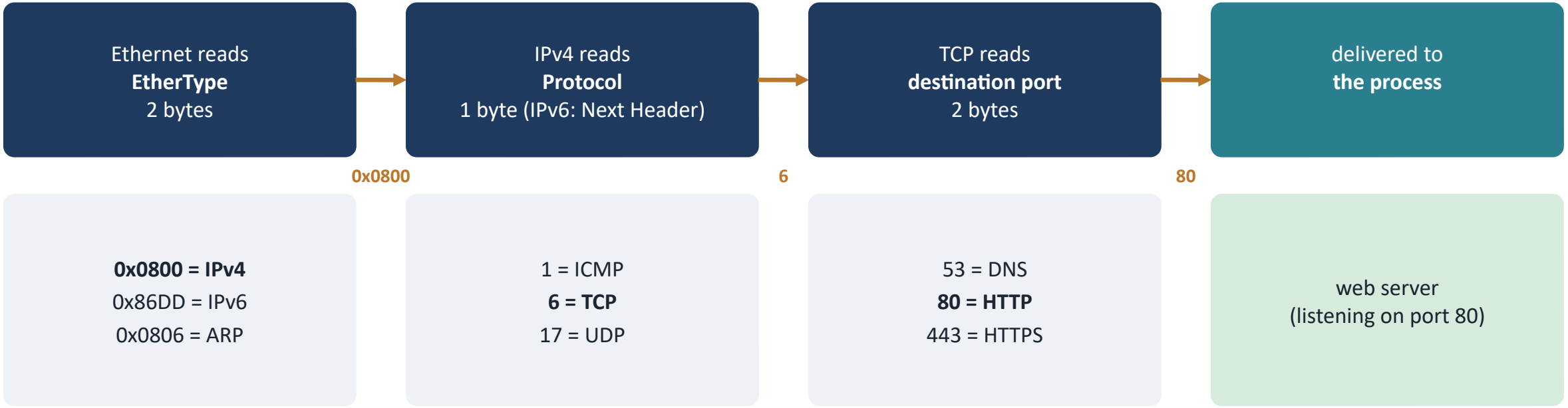


1 The unit received from above = **payload**, in OSI (Open Systems Interconnection) terms an **SDU (service data unit)**.

2 Payload + this layer's header = this layer's **PDU (protocol data unit)**: segment, datagram, frame.

Key idea Going down, every layer adds its header; going up, every layer removes its own and passes the rest on.

3 · Demultiplexing: one header field picks the next receiver



1 **EtherType → Protocol → port**: one field per layer chooses the receiver above.

2 Bold = this example: a frame carrying IPv4, carrying TCP, for port 80.

Key idea Three fields deliver a frame to the right process: EtherType, IP Protocol, destination port.

HTTPS (HyperText Transfer Protocol Secure) · HTTP (HyperText Transfer Protocol) · ICMP (Internet Control Message Protocol) · IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · DNS (Domain Name System) · ARP (Address Resolution Protocol) · IP (Internet Protocol)

3 · Port numbers: three ranges, one registry

0 65535

well-known 0 – 1023	registered 1024 – 49151	dynamic 49152 – 65535
standard services: 53 DNS · 80 HTTP · 443 HTTPS	particular applications, e.g. 8080 (alternative HTTP)	chosen by the operating system for clients, e.g. 51234

IANA
Internet Assigned Numbers Authority
keeps the public registries: port numbers, IP protocol numbers, …

IEEE
Institute of Electrical and Electronics Engineers
assigns the EtherType values (link layer)

Key idea Servers listen on well-known ports; the operating system gives each client socket a dynamic one.

HTTPS (HyperText Transfer Protocol Secure) · HTTP (HyperText Transfer Protocol) · DNS (Domain Name System) · IP (Internet Protocol)

3.1 · A 400-byte web request leaves the card as 458 bytes

Ethernet header 14 B · link	**IPv4 header** 20 B · network	**TCP header** 20 (+ options) B · transport	**HTTP request** 400 B · application	**trailer** 4 B · link
dest MAC · source MAC · EtherType 0x0800	source & dest IP · TTL · Protocol = 6 · header checksum	source port 51234 · dest port 80 · sequence & ACK numbers	`GET /index.html HTTP/1.1` + header lines	FCS, CRC-32

to scale:



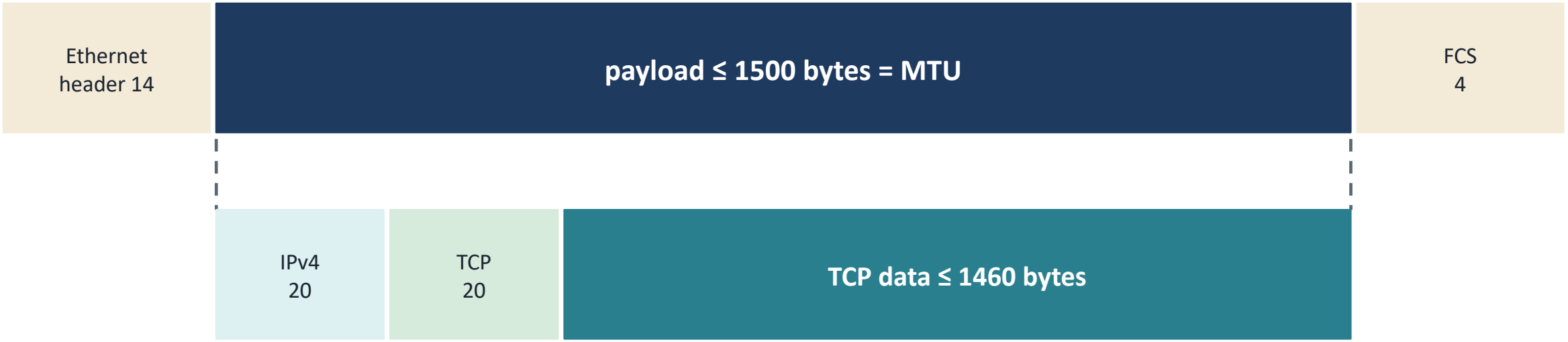
58 bytes of headers + 400 bytes of HTTP = 458 bytes; 58 / 458 ≈ 13 % overhead

MAC = Media Access Control · TTL = time to live · FCS = frame check sequence · CRC = cyclic redundancy check

Key idea One request, three layers of wrapping: 14 + 20 + 20 bytes in front, 4 bytes behind.

HTTP (HyperText Transfer Protocol) · IPv4 (Internet Protocol version 4) · TCP (Transmission Control Protocol) · ACK (acknowledgement) · IP (Internet Protocol)

3.1 · The MTU: why 1460 appears everywhere



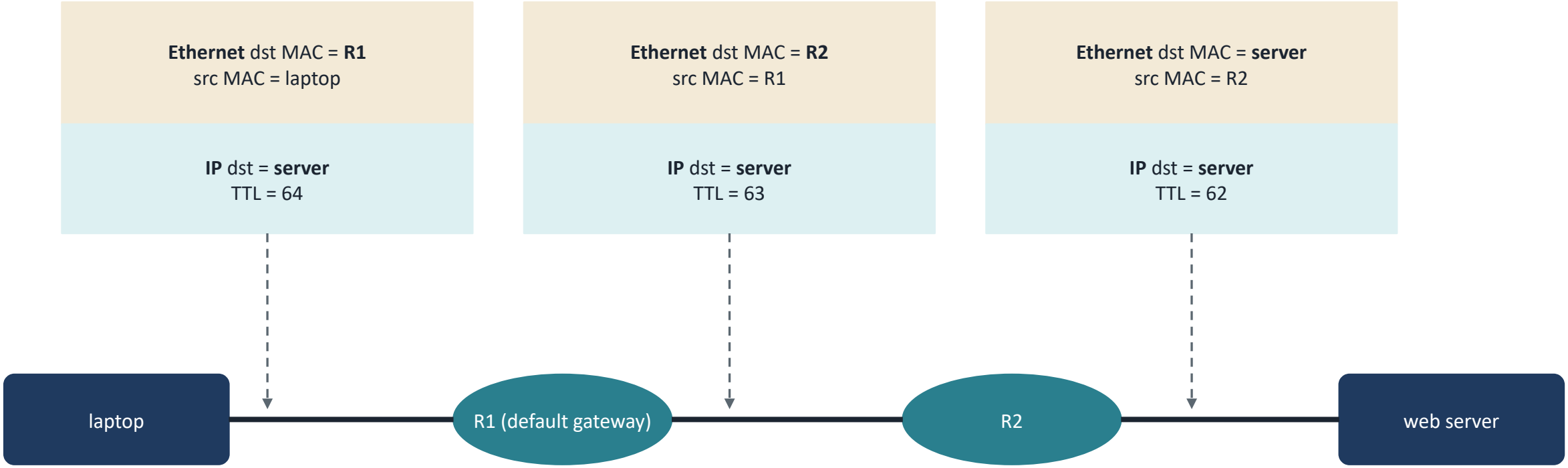
over IPv4: 1500 − 20 − 20 = **1460**
the usual MSS (maximum segment size)

over IPv6: 1500 − 40 − 20 = **1440**
IPv6 header is 40 bytes

Key idea MTU 1500 minus the IP and TCP headers: at most 1460 bytes of data per Ethernet frame over IPv4.

IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · TCP (Transmission Control Protocol) · MTU (maximum transmission unit) · FCS (frame check sequence) · IP (Internet Protocol)

3.1 · Where the destination MAC address points



1 **MAC** (Media Access Control) addresses: next hop on this link — rewritten at every router.

2 **IP header**: end to end; only the **TTL** (time to live) drops by one per router.

Key idea IP address = final destination; MAC address = next hop. You will check this yourself in Laboratory 2.

IP (Internet Protocol)

4.1 · The OSI reference model: seven layers (ISO 7498, 1984)

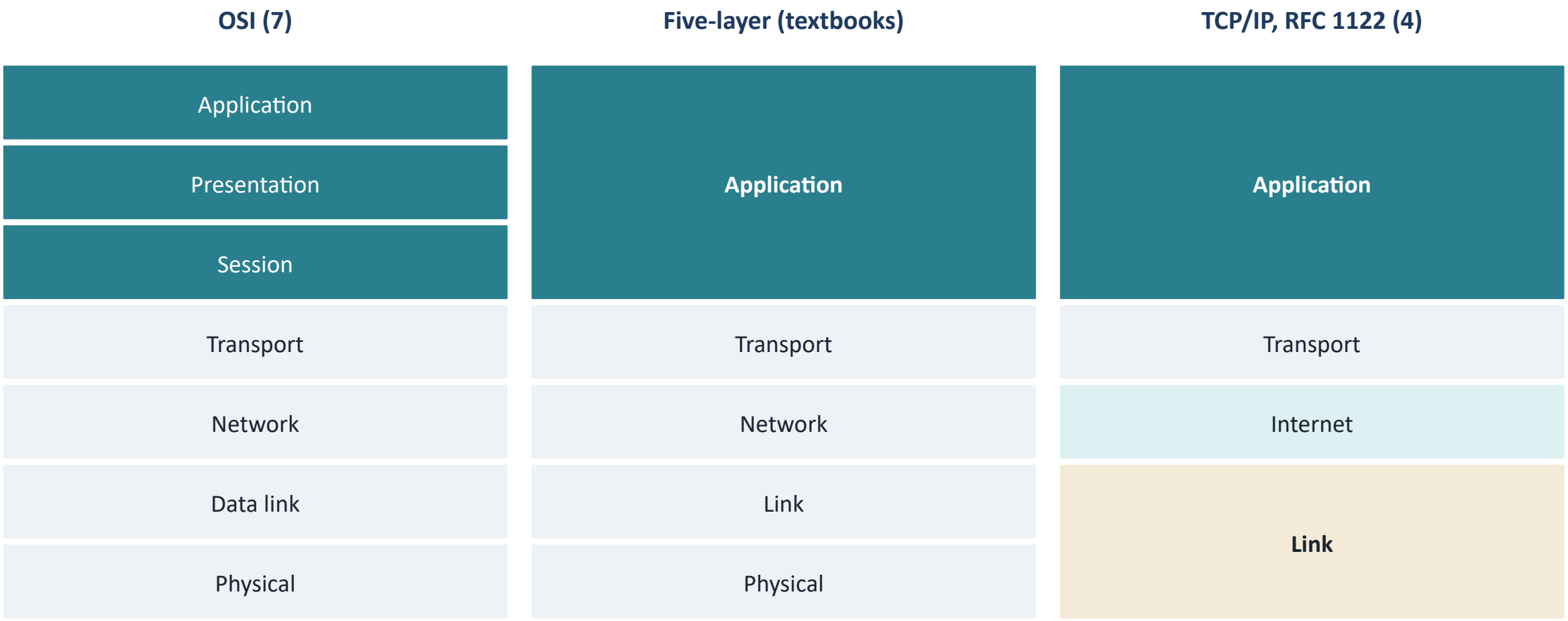
#	OSI layer	Function	Where it lives in the Internet
7	**Application**	services used directly by applications	HTTP, SMTP, DNS
6	**Presentation**	representation: encoding, compression, encryption	inside applications and libraries: UTF-8, JSON, TLS
5	**Session**	dialogue control, synchronisation, checkpoints	inside applications: HTTP sessions, TLS resumption
4	**Transport**	end-to-end delivery between processes	TCP, UDP, QUIC
3	**Network**	routing and forwarding across networks	IP
2	**Data link**	frames between neighbours; error detection	Ethernet, Wi-Fi
1	**Physical**	raw bits over a medium	signalling on copper, fibre, radio

Sand rows: layers with no separate protocol in the Internet — their work is done inside applications.

Key idea Bottom up: “Please Do Not Throw Sausage Pizza Away” — Physical, Data link, Network, Transport, Session, Presentation, Application.

UTF-8 (Unicode Transformation Format, 8-bit) · HTTP (HyperText Transfer Protocol) · QUIC (a transport protocol over UDP, User Datagram Protocol; first named Quick UDP Internet Connections) · SMTP (Simple Mail Transfer Protocol) · JSON (JavaScript Object Notation) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · TLS (Transport Layer Security) · DNS (Domain Name System) · OSI (Open Systems Interconnection) · ISO (International Organization for Standardization) · IP (Internet Protocol)

4.2 · Three models side by side



Key idea The Internet folds presentation and session into applications, and link and physical into one link layer.

TCP (Transmission Control Protocol) · OSI (Open Systems Interconnection) · RFC (Request for Comments) · IP (Internet Protocol)

4.2 · Why TCP/IP won — and what OSI left behind

TCP/IP	protocols running on real machines →	shipped free with Berkeley UNIX, early 1980s →	**the network of the world**
OSI	model by committee, late 1970s →	protocols late, complex, expensive →	**protocols faded**

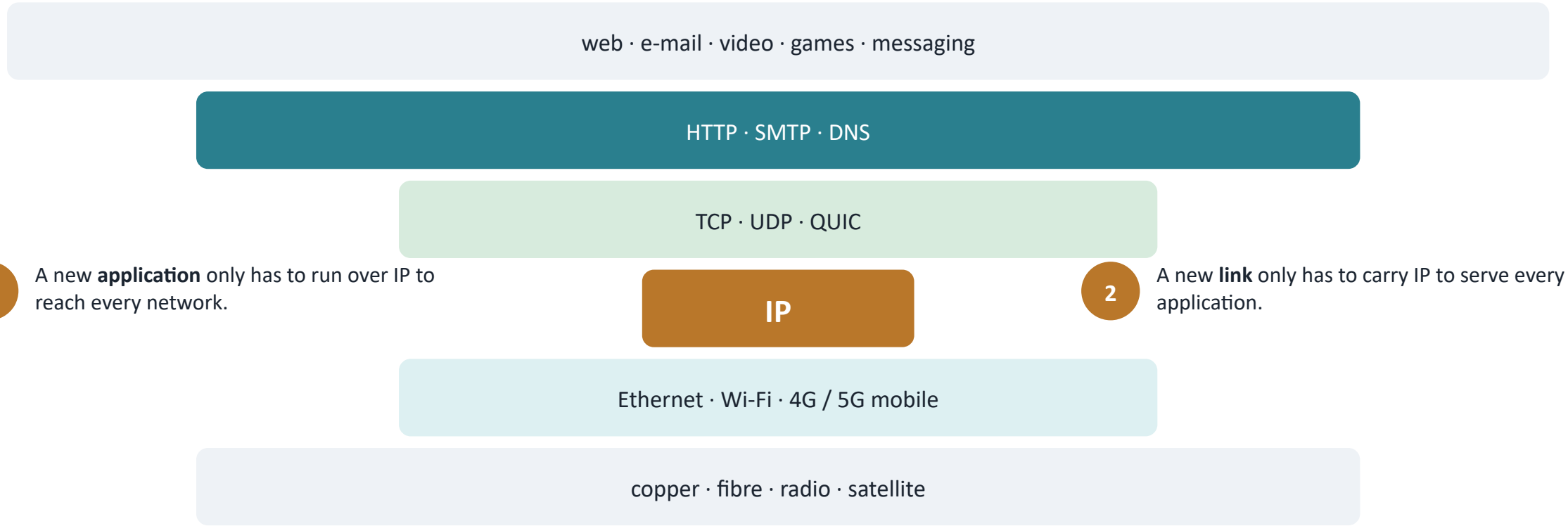
What survives of OSI: the vocabulary

- “a layer-2 switch”
- “a layer-3 router”
- “a layer-7 load balancer”

Key idea TCP/IP won with running code; OSI survives as the words we use to number layers.

TCP (Transmission Control Protocol) · OSI (Open Systems Interconnection) · IP (Internet Protocol)

4.3 · The narrow waist: an hourglass with IP in the middle



1 A new **application** only has to run over IP to reach every network.

2 A new **link** only has to carry IP to serve every application.

Cost: the waist is very hard to change — the move from IPv4 to IPv6 has taken more than twenty-five years.

Key idea Everything runs over IP and IP runs over everything: easy to grow at the edges, hard to change in the middle.

HTTP (HyperText Transfer Protocol) · QUIC (a transport protocol over UDP, User Datagram Protocol; first named Quick UDP Internet Connections) · SMTP (Simple Mail Transfer Protocol) · IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · DNS (Domain Name System) · IP (Internet Protocol) · 4G (fourth generation) · 5G (fifth generation)

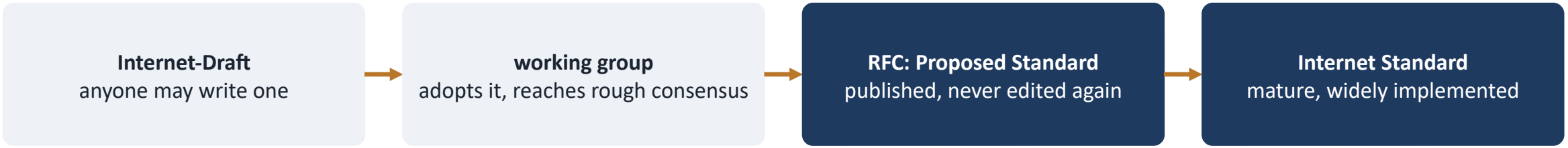
5 · Who makes the rules

protocols	**IETF** Internet Engineering Task Force	IP, TCP, UDP, QUIC, DNS, HTTP, TLS … open to anyone; work in public mailing lists and working groups
local links	**IEEE 802** committees Institute of Electrical and Electronics Engineers	802.3 Ethernet · 802.11 Wi-Fi · 802.1 bridging and virtual LANs (local area networks) — local and metropolitan networks
telecom	**ITU-T** and **3GPP** International Telecommunication Union – Telecommunication Standardization Sector · 3rd Generation Partnership Project	telecommunications standards; 3GPP specifies the 4G (fourth generation) and 5G (fifth generation) mobile systems
the Web	**W3C** and **WHATWG** World Wide Web Consortium · Web Hypertext Application Technology Working Group	HTML (HyperText Markup Language), CSS (Cascading Style Sheets), browser APIs (application programming interfaces)
numbers	**IANA** and the five **RIRs** Internet Assigned Numbers Authority · Regional Internet Registry (one per region)	IANA gives address blocks to the RIRs; Romania → **RIPE NCC** (Réseaux IP Européens Network Coordination Centre)

Key idea IETF writes the Internet's protocols, IEEE its local links, IANA and the RIRs hand out its numbers.

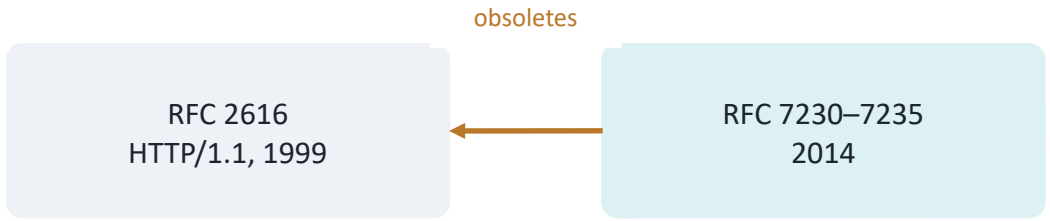
HTTP (HyperText Transfer Protocol) · QUIC (a transport protocol over UDP, User Datagram Protocol; first named Quick UDP Internet Connections) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · TLS (Transport Layer Security) · DNS (Domain Name System) · IP (Internet Protocol)

5 · From Internet-Draft to Internet Standard



the two levels of the standards track since RFC 6410

RFCs are never edited — they are replaced



Not every RFC is a standard

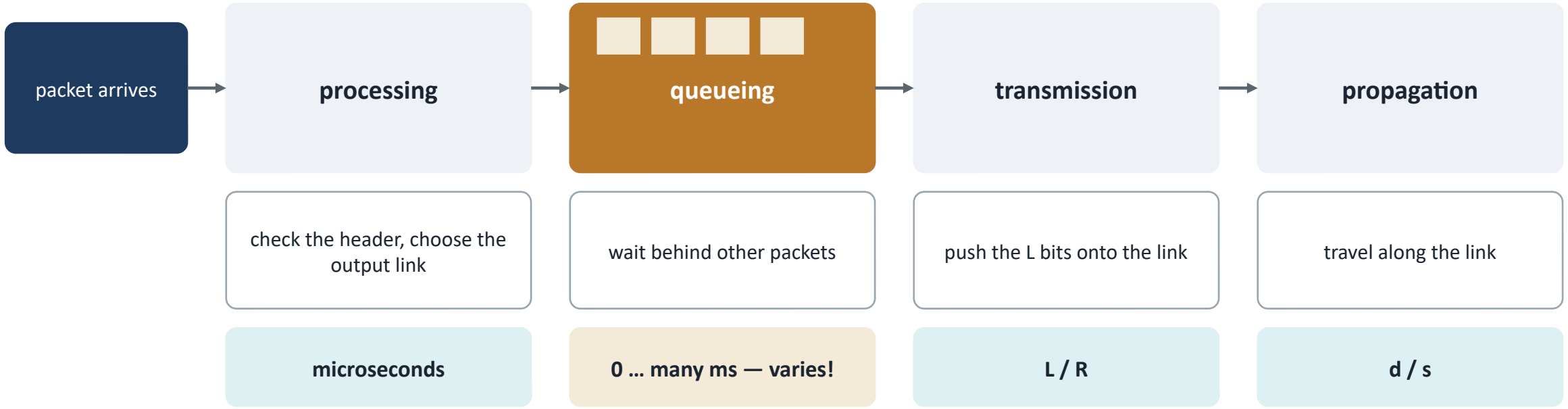
- informational
- experimental
- historic
- 1 April: jokes

> “We reject kings, presidents and voting. We believe in rough consensus and running code.”
> — David Clark, 1992

Key idea A specification is taken seriously when independent implementations exist and work together.

HTTP (HyperText Transfer Protocol) · RFC (Request for Comments)

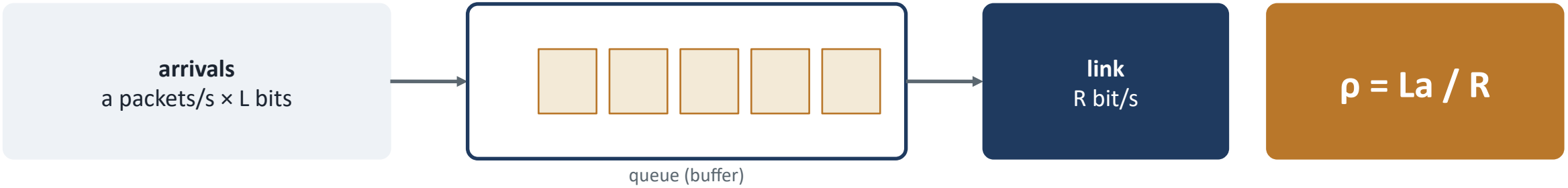
6 · Four delays at a router — only one is unpredictable



1 L = packet length (bits) · R = link rate (bit/s) · d = link length · s = propagation speed. Queueing is the interesting one: it depends on **other people's** traffic.

Key idea Three delays are formulas; queueing delay is statistics — the subject of this section.

6.1 · Traffic intensity ρ = La / R



ρ > 1

bits arrive faster than they leave: the queue grows **without limit**; buffers are finite → **loss**. No buffer size fixes this.

ρ ≤ 1, regular arrivals

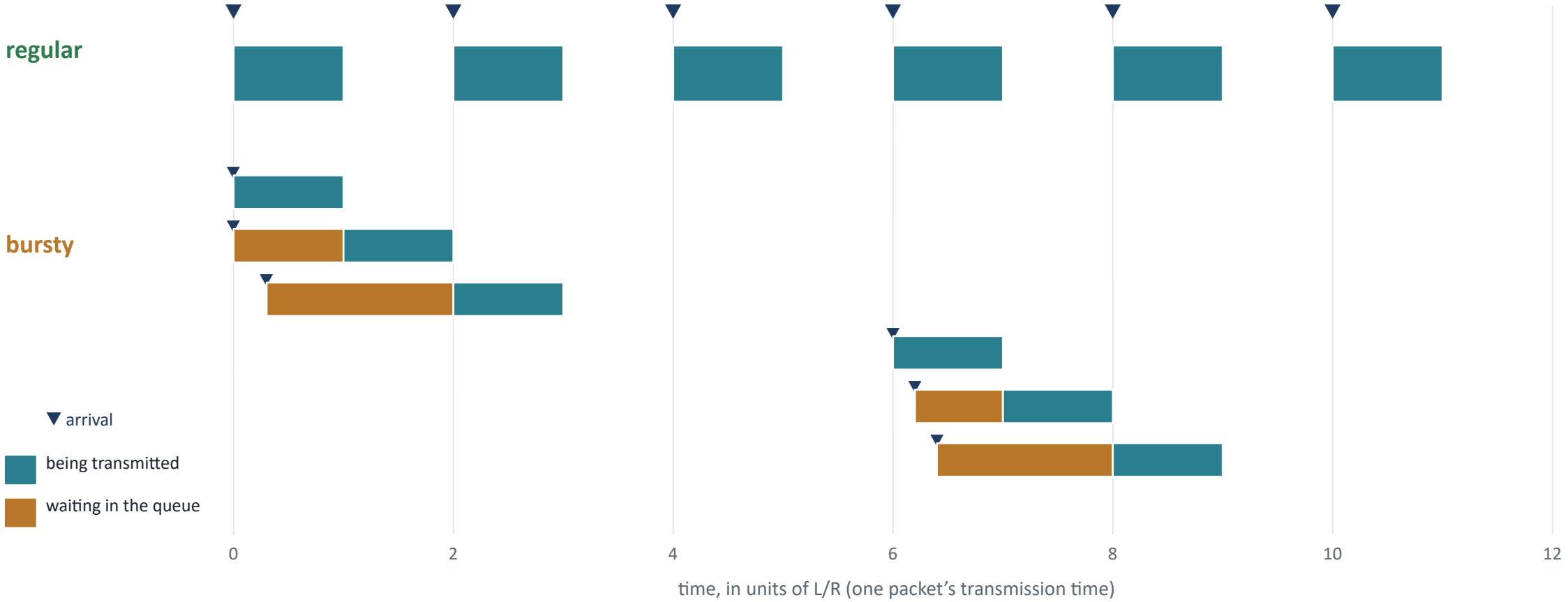
one packet every L/R seconds: **no packet ever waits**.

ρ ≤ 1, random bursty arrivals

the real case: packets arrive in bunches and **wait**; the closer ρ is to 1, the longer and more often.

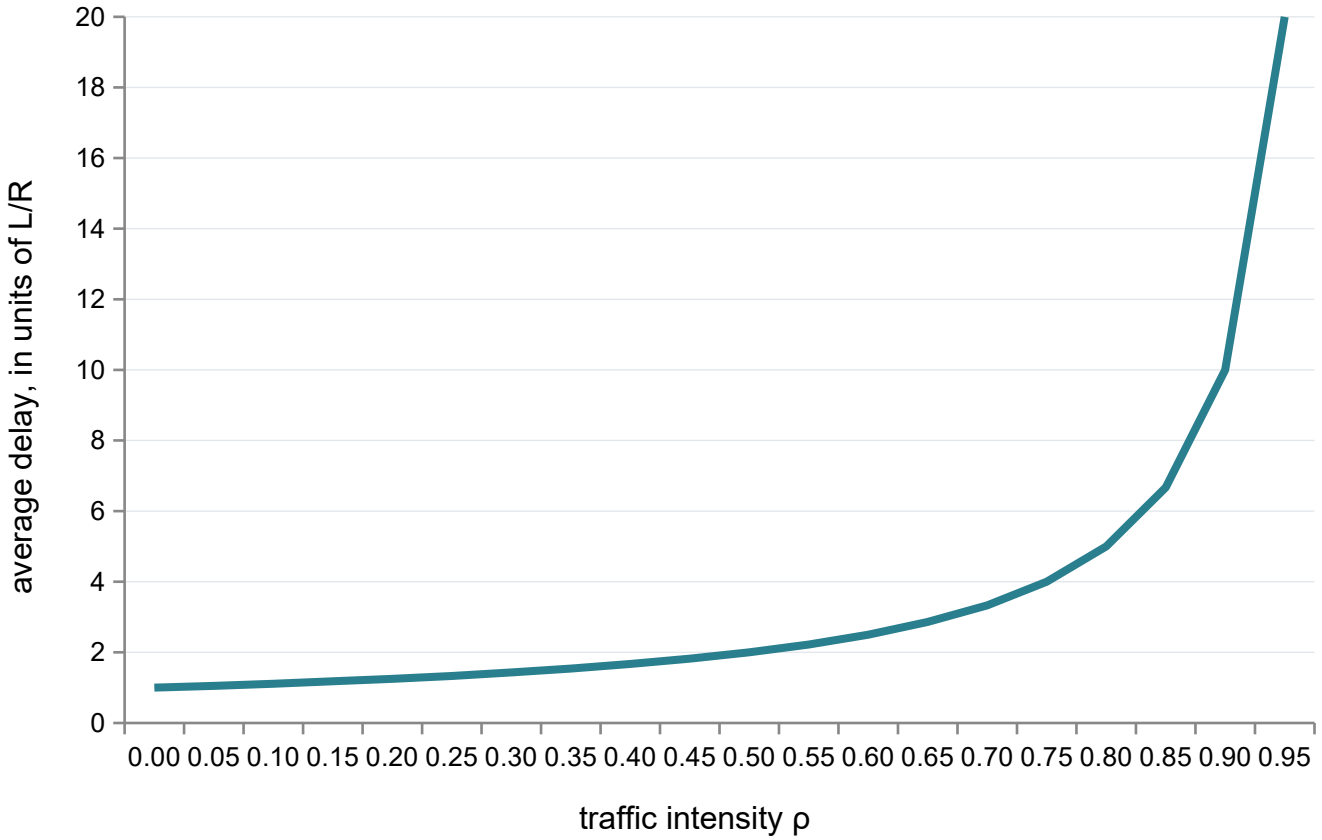
Key idea Rule one of traffic engineering: design so that ρ stays below 1.

6.1 · Same load, very different waits



Key idea Queues come from burstiness: with random arrivals, waits appear long before the link is full.

6.2 · How fast delay grows: the M/M/1 model



$$d = (L/R) / (1 - \rho)$$

load ρ	delay
50 %	2 × L/R
80 %	5 × L/R
90 %	10 × L/R
95 %	20 × L/R

1 90 % → 95 %: **+5 % traffic doubles the delay**. Operators upgrade long before “full”.

Key idea Real traffic is not M/M/1 — trust the shape, not the numbers: delay explodes as ρ approaches 1.

M/M/1 (Markovian arrivals / Markovian service times / 1 server)

6.3 · Loss, buffer size and bufferbloat

Tail drop



buffer: normal, full

Queue full → arriving packets are **dropped**. TCP retransmits them (extra delay); without TCP they are simply missing (glitches).

Bufferbloat (≈ 2010)



buffer: very large, full

Huge, full buffer = a long queue. One big upload adds **hundreds of ms** to every packet — including a video call's.

Active queue management



buffer: normal, almost empty

Algorithms such as **CoDel** (Controlled Delay) drop or mark packets **early**, keeping the standing queue short.

Key idea A big full buffer is just a long queue: the cure is managing the queue, not only sizing it. (Semester 2 lab.)

TCP (Transmission Control Protocol)

7.1 · End-to-end throughput: the bottleneck decides

throughput = min(R_1, R_2, …, R_n)



A shared link is divided between flows

1 Often the **access links** limit; a congested core can too.



10 transfers

Key idea Throughput = the smallest rate this flow can get on any link of its path, shared links included.

7.2 · The bandwidth–delay product: bits in flight

window ≥ BDP: the pipe is full — data forward (top), ACKs (acknowledgements) back (bottom)



1 Gbit/s × 30 ms (Bucharest–Frankfurt and back) = 30 Mbit ≈ **3.75 MB** in flight

window = 64 KB (the original TCP limit): mostly empty



64 KB every 30 ms ≈ **17.5 Mbit/s** on a 1 Gbit/s path

BDP = R × RTT — to keep the path busy, a sender must be allowed one BDP of unacknowledged data

Key idea To fill a path, a sender needs at least one bandwidth–delay product of data in flight — we return to it with TCP windows.

TCP (Transmission Control Protocol) · RTT (round-trip time)

7 · Answer to the opening question

1 MB: a few round trips to set up and ramp up



e.g. 3 RTT × 10 ms = 30 ms



3 RTT × 300 ms = 900 ms

you finish first

10 GB: the bottleneck rate decides



10 GB at 100 Mbit/s = 800 s



10 GB at 1 Gbit/s ≈ 80 s

neighbour wins, ×10

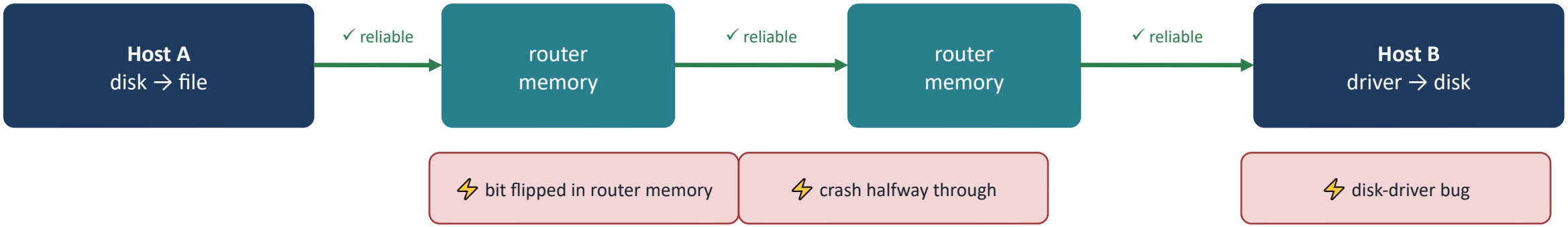
1 Neighbour's 80 s needs a window covering BDP = 1 Gbit/s × 300 ms = **37.5 MB**.

Key idea Which link is "faster" depends on the size of what you send: delay rules small transfers, throughput rules large ones.

RTT (round-trip time) · BDP (bandwidth–delay product)

8 · The end-to-end argument: careful file transfer

A function can be implemented completely and correctly only with the knowledge and help of the application at the end points. Inside the network it is never **sufficient** — only a **performance enhancement**.



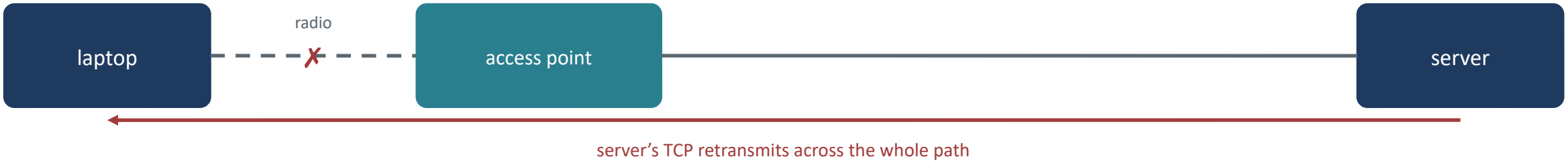
end-to-end check: checksum computed by A over the file, verified by B — catches all of these

1. Once the end-to-end check exists, perfect hop-by-hop reliability is **not needed for correctness**.

Key idea Only the endpoints can check what the endpoints care about: the network may help, but cannot be relied upon.

8 · When the network should help anyway: Wi-Fi

(a) recovery left to TCP alone — the loss is noticed and repaired across the whole path



costs at least one end-to-end round trip — tens to hundreds of ms

(b) Wi-Fi retransmits locally, then TCP still checks end to end



repaired within microseconds, before TCP even notices

Key idea Hop-by-hop recovery is a performance enhancement; correctness still comes from the end-to-end check.

TCP (Transmission Control Protocol)

8 · Consequences in the Internet's design



Middleboxes break the principle — they inspect or change what passes:

NAT box | firewall | transparent proxy → new, unknown transport protocol → **blocked** ✗

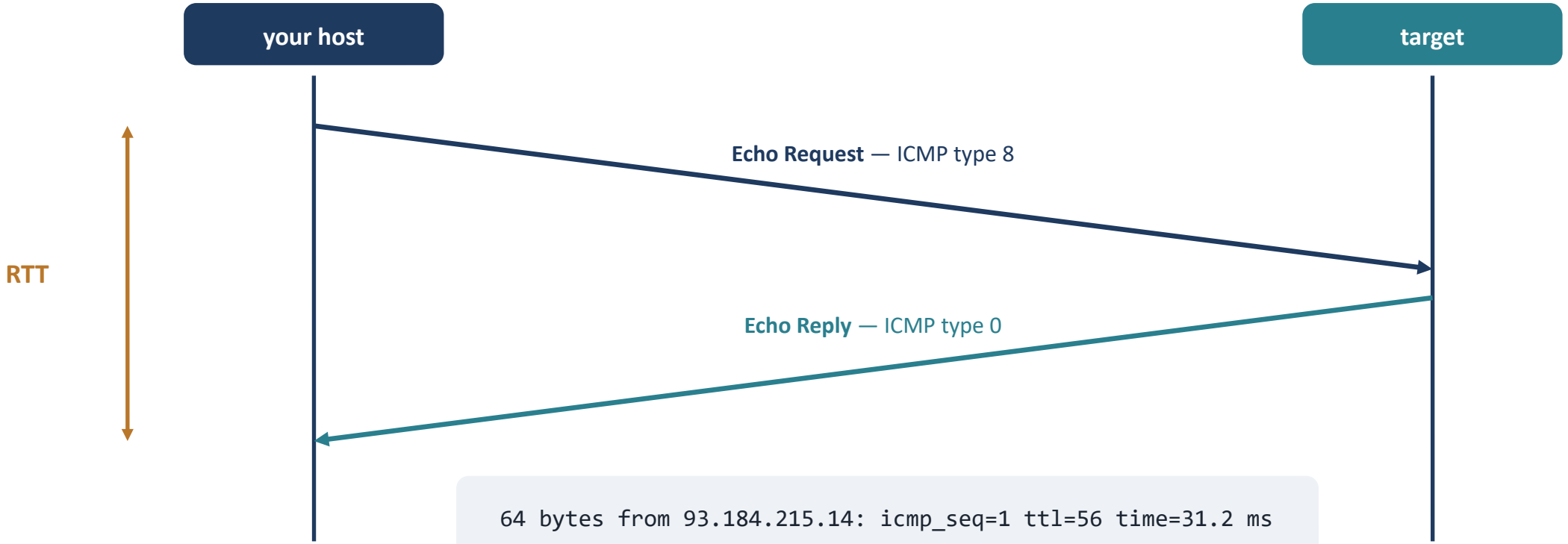
NAT = network address translation

QUIC: looks like UDP, headers encrypted → **passes** ✓

Key idea Intelligence at the edges, simplicity in the core — middleboxes that break this make the Internet hard to change.

QUIC (a transport protocol over UDP, User Datagram Protocol; first named Quick UDP Internet Connections) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · TLS (Transport Layer Security) · IP (Internet Protocol)

9 · How ping measures one round trip

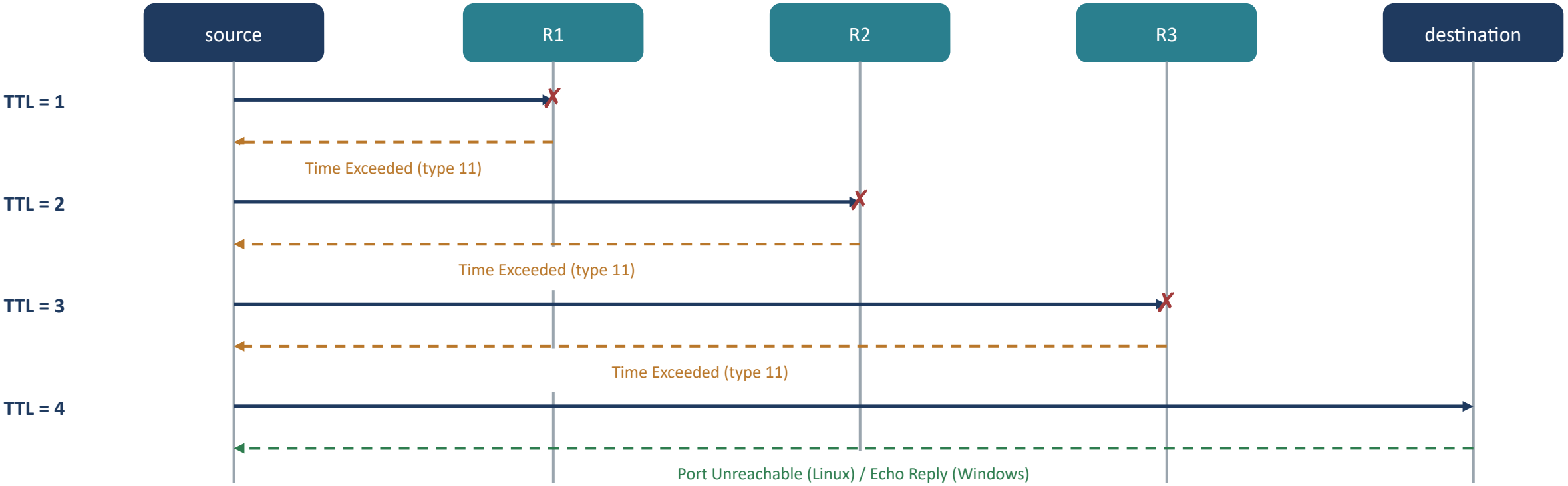


1 Includes **transmission, propagation, queueing and processing — in both directions**. ICMP = Internet Control Message Protocol.

Key idea One ping = one Echo Request + one Echo Reply = one round trip, all four delays included.

RTT (round-trip time)

9 · traceroute: a tool built from the TTL field



Linux traceroute: UDP probes to unlikely ports from 33434 up
destination answers Port Unreachable

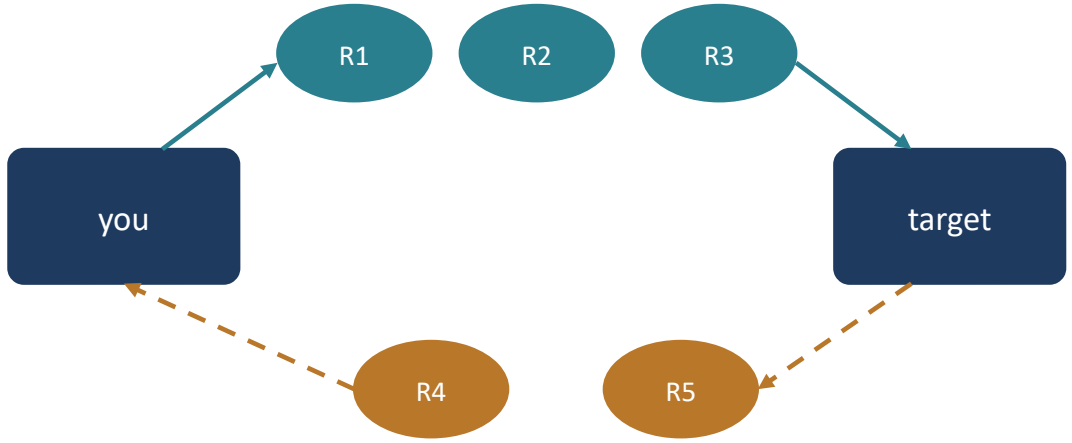
Windows tracert: ICMP Echo Request probes
destination answers Echo Reply

Key idea Van Jacobson, 1987: a whole tool built from a side effect of a header field meant for something else.

ICMP (Internet Control Message Protocol) · UDP (User Datagram Protocol) · TTL (time to live)

9 · Two cautions when reading the numbers

1 · Paths can be asymmetric



Replies may come back by another route: an RTT cannot tell on which side the delay lies.

2 · Routers answer last

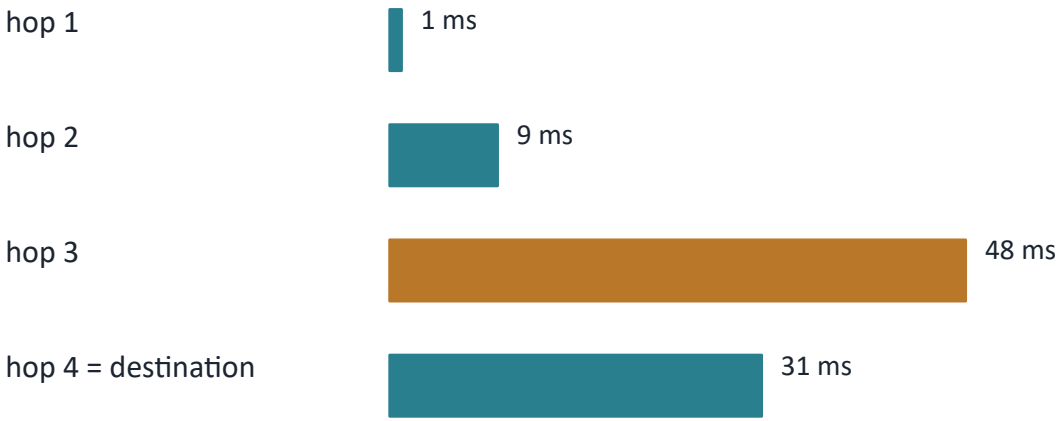


Illustration: hop 3 generates its ICMP replies slowly, so it looks slower than the destination behind it.

Key idea Read traceroute with care: a slow middle hop may just be a busy router answering ICMP last.

ICMP (Internet Control Message Protocol) · RTT (round-trip time)

Summary of Lecture 2

How the work is divided

Layers
service · interface · protocol · peers

→

Encapsulation
headers down, demultiplexing up: EtherType, Protocol, port

→

Models
OSI 7 · TCP/IP 4 (5) · IP is the narrow waist

→

Standards
IETF writes RFCs · IANA registers · RIPE NCC for Romania

How well it works

Queues
ρ = La/R; above 1 unbounded; delay explodes near 1; bufferbloat

→

Throughput
the bottleneck, shares included · BDP = R × RTT

→

End to end
functions needing the endpoints belong in the endpoints

Key idea Structure and performance: layers make the network buildable; queues, bottlenecks and the endpoints decide how well it works.

RIPE NCC (Réseaux IP Européens Network Coordination Centre) · IETF (Internet Engineering Task Force) · IANA (Internet Assigned Numbers Authority) · TCP (Transmission Control Protocol) · RTT (round-trip time) · OSI (Open Systems Interconnection) · BDP (bandwidth–delay product) · RFC (Request for Comments) · IP (Internet Protocol)

Review questions: where each answer is drawn

1 a service, interface and protocol at another layer

→ 2 · Service, interface, protocol

2 which protocols does the router take part in?

→ 2.1 · Peer communication

3 three fields that deliver a frame to a browser

→ 3 · Demultiplexing

4 why the destination MAC changes and the IP does not

→ 3.1 · Where the MAC points

5 OSI layers with no TCP/IP counterpart

→ 4.2 · Three models

6 why a narrow waist eases apps but freezes IP

→ 4.3 · Narrow waist

7 “60 % used — 40 % spare, no upgrade needed”?

→ 6.2 · M/M/1 curve

8 why a bigger buffer is not a cure for loss

→ 6.3 · Bufferbloat

9 the end-to-end argument + one network help

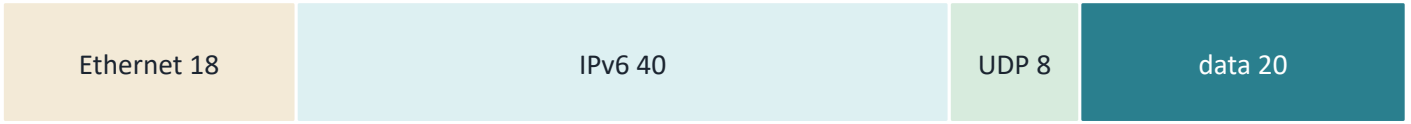
→ 8 · End to end, Wi-Fi

Key idea Answer each question by redrawing its scheme from memory.

M/M/1 (Markovian arrivals / Markovian service times / 1 server) · TCP (Transmission Control Protocol) · OSI (Open Systems Interconnection) · MAC (Media Access Control) · IP (Internet Protocol)

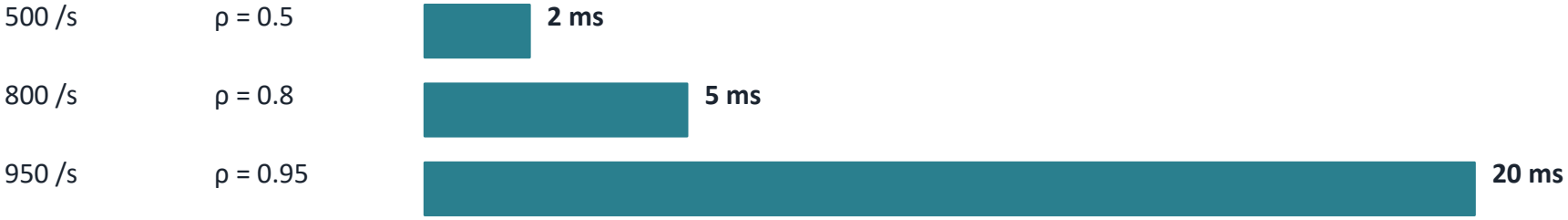
Exercises 1–3, solved as schemes

1 · 20-byte message over UDP and IPv6 on Ethernet

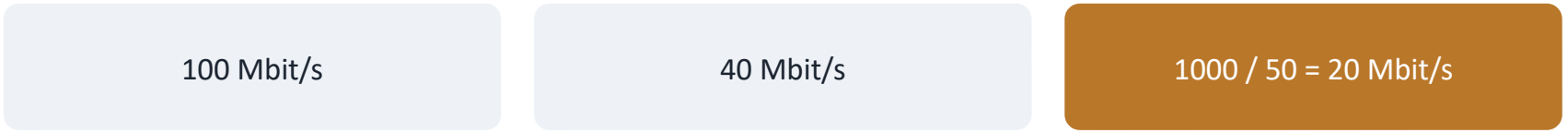


86 bytes per frame → **20 / 86 ≈ 23 %** data

2 · 1000-byte packets, 8 Mbit/s link: L/R = 1 ms, ρ = 1 at 1000 packets/s



3 · links of 100 Mbit/s, 40 Mbit/s and 1 Gbit/s shared by 50 transfers



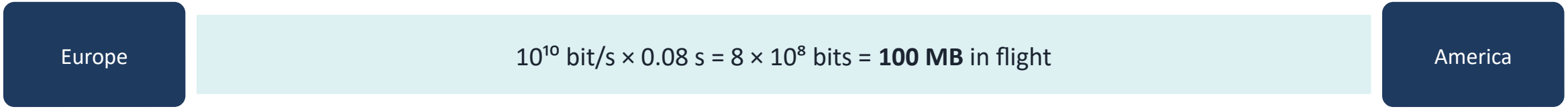
→ min = **20 Mbit/s**

Key idea Draw the frame, the curve, the path — and the arithmetic follows.

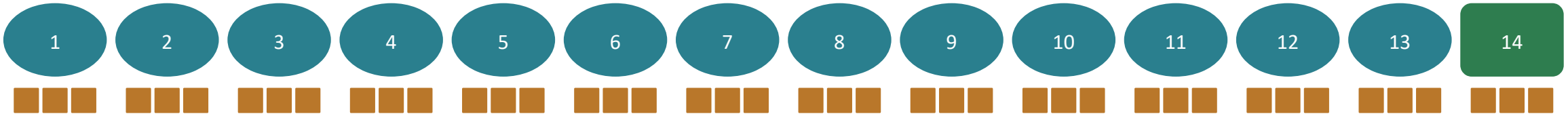
IPv6 (Internet Protocol version 6) · UDP (User Datagram Protocol)

Exercises 4–5, solved as schemes

4 · 10 Gbit/s transatlantic path, RTT 80 ms



5 · traceroute, 3 probes per hop, destination 14 hops away



14 × 3 = 42 probes
hops 1–13 answer **Time Exceeded**

hop 14 (destination) answers
Port Unreachable (Linux) or Echo Reply (Windows tracert)

Key idea BDP = rate × round trip; a traceroute costs probes × hops, and only the last hop answers differently.

RTT (round-trip time) · BDP (bandwidth–delay product)

Reading, laboratory and next lecture

Required

Kurose & Ross, *Computer Networking: A Top-Down Approach*, 8th ed., ch. 1: delay, loss, throughput; protocol layers

Recommended

Saltzer, Reed, Clark, “End-to-End Arguments in System Design”, ACM (Association for Computing Machinery) *Transactions on Computer Systems*, 1984

Peterson & Davie, ch. 1 · RFC 1122, § 1.1 · Gettys & Nichols, “Bufferbloat”, 2012

Laboratory 2

capture your own web request, open every header, check MAC and IP addresses, measure an RTT from the TCP handshake

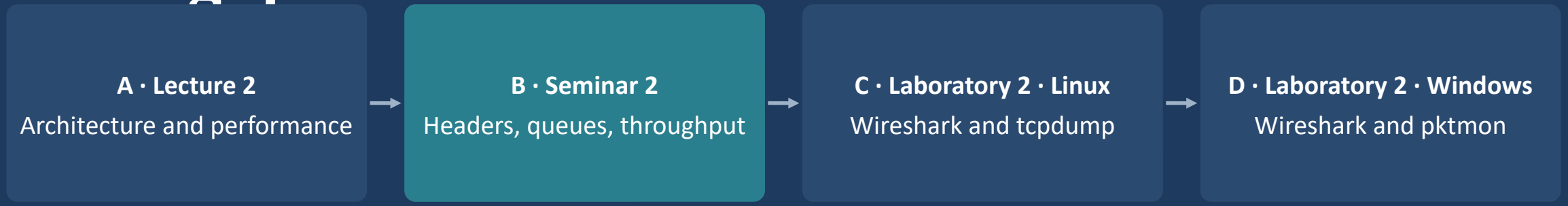
Next lecture

the application layer: architectures, sockets, the Web and HTTP/1.1

Key idea Read chapter 1, then see every scheme of this lecture again in Wireshark.

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol) · RTT (round-trip time) · MAC (Media Access Control) · RFC (Request for Comments) · IP (Internet Protocol)

Part B · Seminar 2: headers, queues and throughput



In this part

A · Problem 1: which layer?

B · Problem 2: decoding a frame by hand

C · Problem 3: how much of the link is useful?

D · Problems 4–5: traffic intensity, Little's law

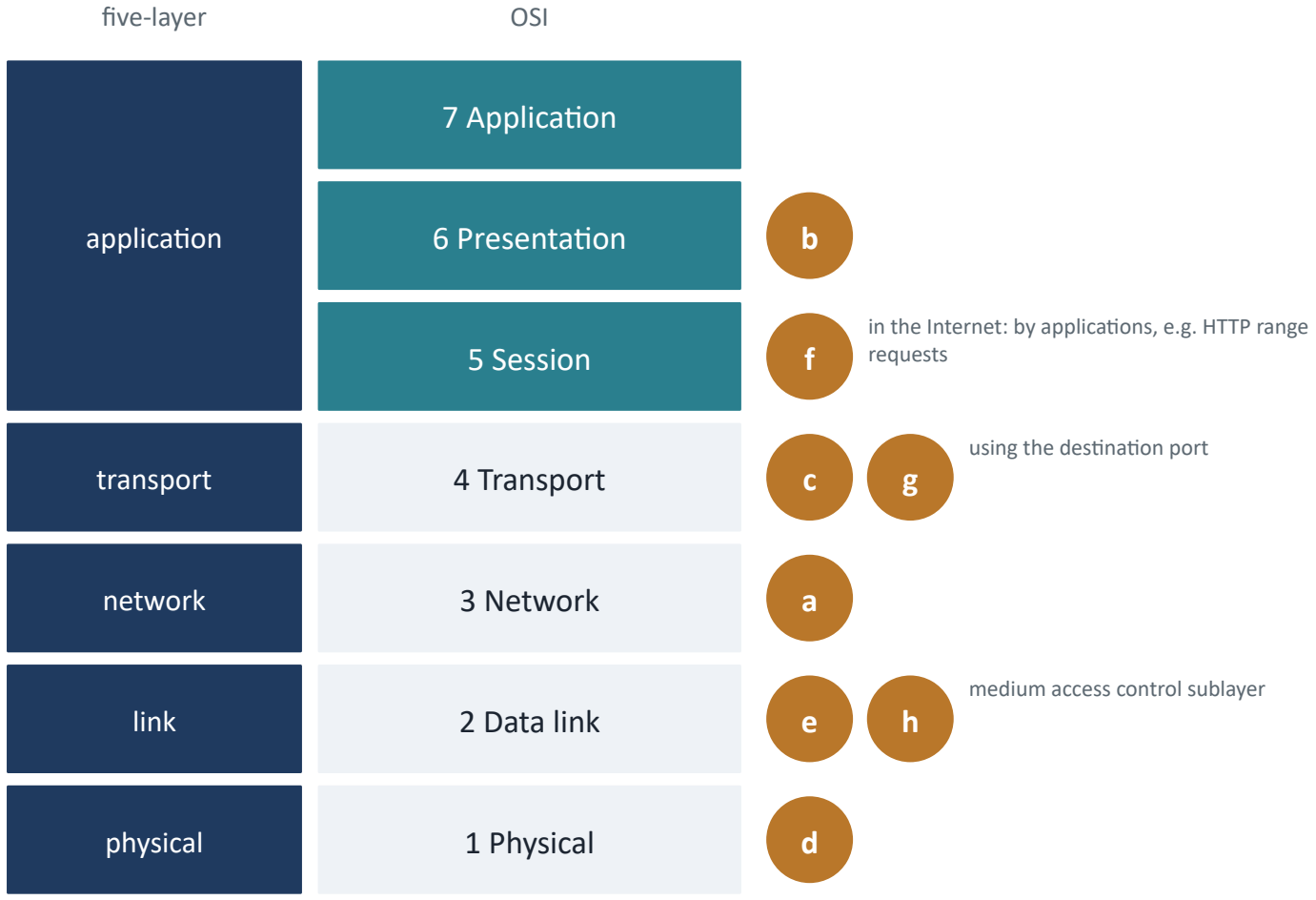
E · Problems 6–7: bottlenecks, bandwidth–delay product

F · Problem 8: the end-to-end argument

Homework

Problem 1 · Which layer? Five-layer model and OSI

- **a** choosing the outgoing link for a packet to another country
- **b** converting a document between character encodings
- **c** retransmitting a segment whose ACK did not arrive
- **d** representing a 1 as a voltage transition
- **e** detecting a corrupted frame on a Wi-Fi link
- **f** resuming a download where it stopped
- **g** delivering a segment to the right process
- **h** deciding which host on a shared cable may send



Key idea Presentation and session work exists in the Internet too — it is simply done inside applications.

HTTP (HyperText Transfer Protocol) · OSI (Open Systems Interconnection) · ACK (acknowledgement)

Problem 2 · A frame in hexadecimal, coloured by header

0000	52	54	00	12	34	01	52	54	00	3a	7f	10	08	00	45	00
0010	00	3d	1c	46	40	00	40	11	06	d5	0a	14	03	39	0a	14
0020	00	35	c8	22	00	35	00	29	96	d2	3b	7a	01	00	00	01
0030	00	00	00	00	00	00	03	77	77	77	07	65	78	61	6d	70
0040	6c	65	03	6f	72	67	00	00	01	00	01					

Ethernet
bytes 0x00–0x0d · 14 B

IPv4
0x0e–0x21 · 20 B

UDP
0x22–0x29 · 8 B

DNS header
0x2a–0x35 · 12 B

DNS question
0x36–0x4a · 21 B

1 Questions: MAC addresses and payload type · IP version, lengths · TTL, protocol, addresses · ports and application · the name looked up · whose MAC?

Key idea Before decoding, cut the bytes into headers: 14 + 20 + 8, and everything after is application data.

IPv4 (Internet Protocol version 4) · UDP (User Datagram Protocol) · DNS (Domain Name System) · MAC (Media Access Control) · TTL (time to live) · IP (Internet Protocol)

Problem 2 · The field map to decode with

Ethernet header (14 bytes)

destination MAC 6 B	source MAC 6 B	EtherType 2 B

UDP header (8 bytes)

source port	destination port
length	checksum

IPv4 header (20 bytes, 4 per row)

ver | hdr len	type of service	total length	
identification		flags + fragment offset	
TTL	protocol	header checksum	
source address			
destination address			
byte 0	byte 1	byte 2	byte 3

first IPv4 byte = two halves
high 4 bits = version · low 4 bits = header length in 32-bit words

Key idea Every field has a fixed size and position — that is what lets hardware and people decode headers.

IPv4 (Internet Protocol version 4) · UDP (User Datagram Protocol) · MAC (Media Access Control) · TTL (time to live)

Problem 2 · Answers 1–3: Ethernet and IPv4 decoded

dest MAC 52:54:00:12:34:01	**source MAC** 52:54:00:3a:7f:10	**08 00** → IPv4

45 v4 · 5 × 4 = 20 B	**00** service	**00 3d** = 61 bytes
1c 46 identification		**40 00** DF (Don't Fragment), offset 0
40 TTL = 64	**11** 17 = UDP	**06 d5** checksum
0a 14 03 39 → source **10.20.3.57**		
0a 14 00 35 → destination **10.20.0.53**		

1. EtherType 0x0800: the payload is an **IPv4** datagram.
2. Total length 61; the frame shown is 75 = **14 + 61** ✓ consistent.
3. TTL 64: a common initial value — the datagram has **not crossed a router** yet.

Key idea 0x45, 0x0800, 0x11: three bytes that tell you IPv4 with a 20-byte header, carried by Ethernet, carrying UDP.

IPv4 (Internet Protocol version 4) · UDP (User Datagram Protocol) · MAC (Media Access Control) · TTL (time to live)

Problem 2 · Answers 4–5: UDP header and the DNS name

c8 22 source port 51234 (dynamic → client)	**00 35** dest port 53 (DNS server)	**00 29** length 41 = 8 + 33	**96 d2** checksum

After 12 bytes of DNS header, the question:

03	77	77	77	07	65	78	61	6d	70	6c	65	03	6f	72	67	00	00 01	00 01
3	w	w	w	7	e	x	a	m	p	l	e	3	o	r	g	end	type A	class IN

→ **www.example.org**

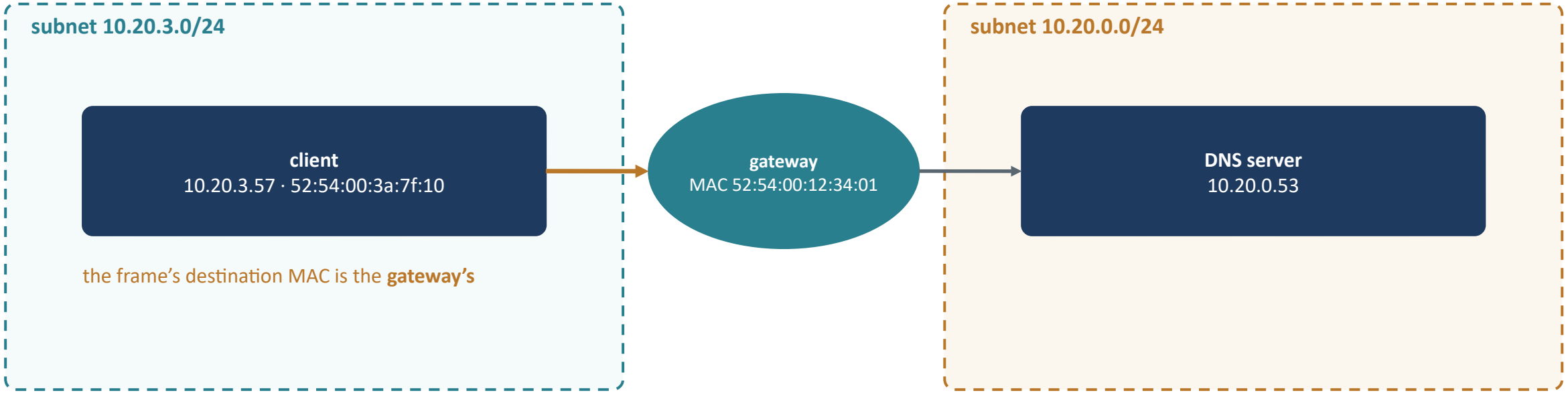
IPv4 address, Internet class

1 Amber bytes are **length counters** (3, 7, 3), then 0 ends the name. 0x77 is the ASCII (American Standard Code for Information Interchange) code of “w”.

Key idea UDP length 41 = 8-byte header + 33 bytes of DNS: 12 of header, 17 of name, 4 of type and class.

IPv4 (Internet Protocol version 4) · UDP (User Datagram Protocol) · DNS (Domain Name System)

Problem 2 · Answer 6: is the destination MAC the server's?



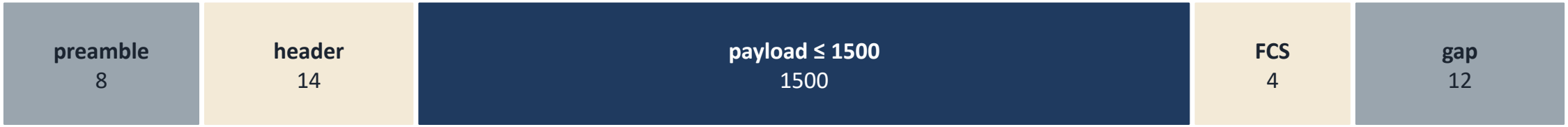
10.20.3.57 vs **10.20.0**.53
first 24 bits differ → not on my link → send to the default gateway

check it
`ip neigh` (Linux) · `arp -a` (Windows)
— exactly what you do in Laboratory 2

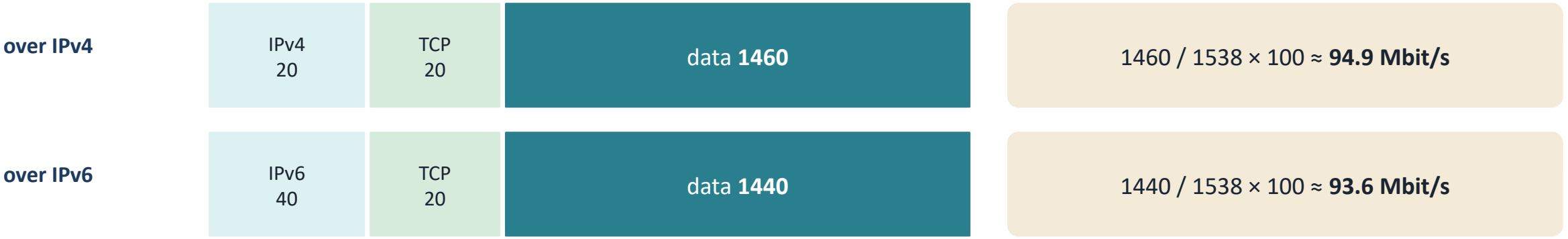
Key idea A frame for another subnet always carries the gateway's MAC — the IP header still names the real destination.

DNS (Domain Name System) · MAC (Media Access Control) · IP (Internet Protocol)

Problem 3 · One full frame occupies 1538 byte-times on the wire



8 + 14 + 1500 + 4 + 12 = 1538 byte-times per maximum-size frame

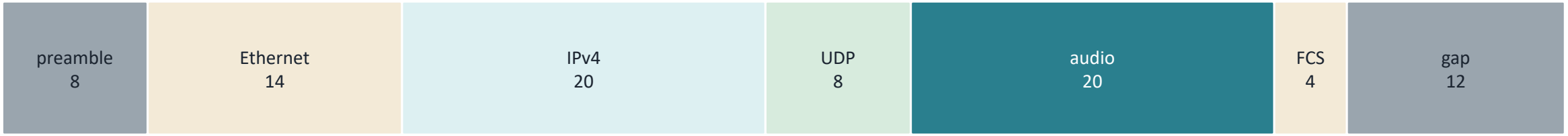


1 Maximum rate of application data on a 100 Mbit/s Ethernet — why speed tests on “100 Mbit/s” links never show 100.

Key idea Framing, headers and gaps take about 5 % of the link even with the largest frames.

IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · TCP (Transmission Control Protocol) · FCS (frame check sequence)

Problem 3 · Q4: tiny voice samples waste the link



86 byte-times on the wire; audio is 20 of them → **20 / 86 ≈ 23 %**

Padding rule

Ethernet pads payloads shorter than 46 bytes up to 46.

Here the payload is 48 bytes: no padding needed.

What real codecs do

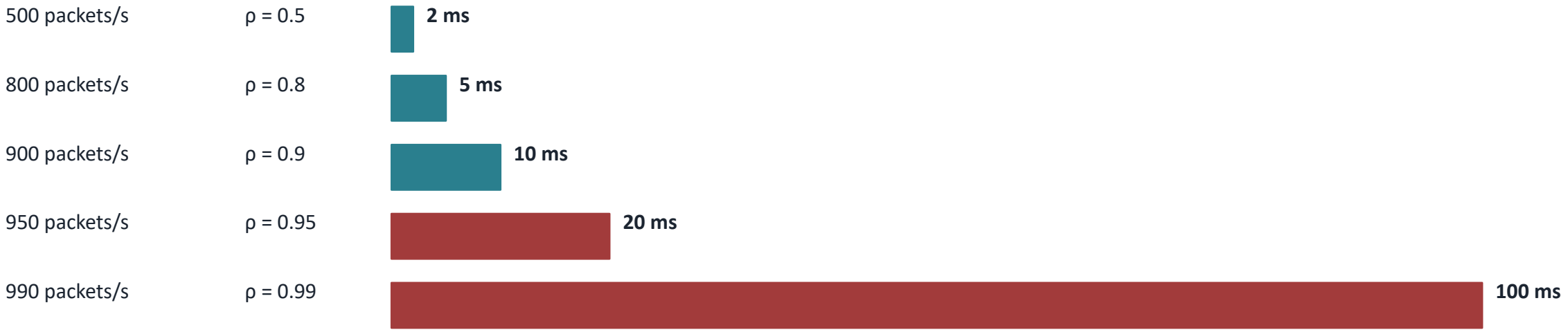
send larger samples less often:

less overhead, at the price of a little extra delay while a sample fills

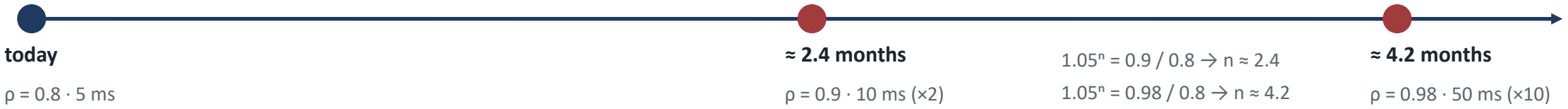
Key idea Small messages pay for every header: three quarters of this link time carries no audio.

IPv4 (Internet Protocol version 4) · UDP (User Datagram Protocol) · FCS (frame check sequence)

Problem 4 · Traffic intensity on a 10 Mbit/s link

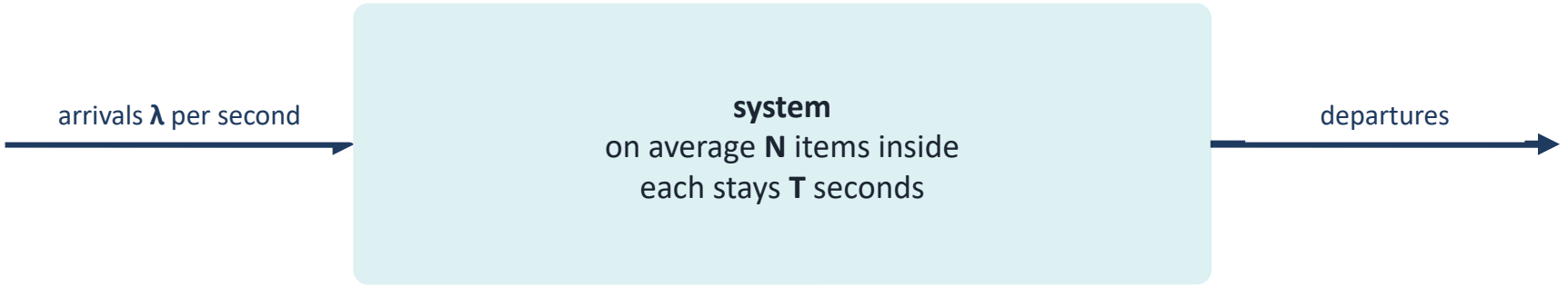


Q3 · traffic grows 5 % a month; the link is at 80 % today



Key idea The operator who waits for “full” waits too long: doubling the delay takes less than three months here.

Problem 5 · Little's law: N = λ × T



1 router buffer

N = 40 packets, λ = 2000 /s

T = N / λ = **20 ms**

2 web server

λ = 300 requests/s, T = 50 ms

N = 300 × 0.05 = **15 requests**

3 TCP connection

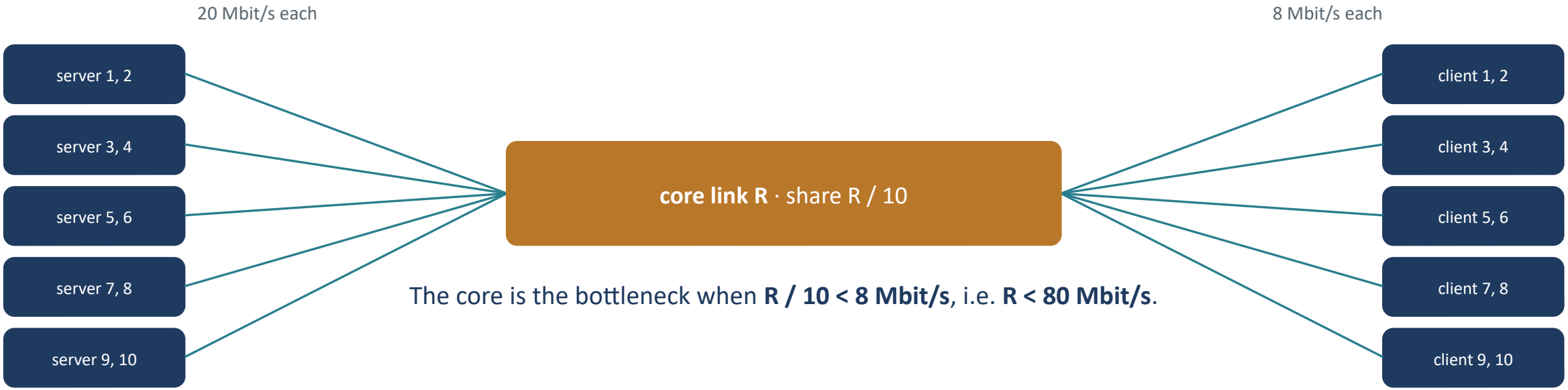
N = 64 KB in flight, T = RTT = 20 ms

rate = N / T ≈ 3.3 MB/s ≈ **26 Mbit/s**

Key idea Little's law links the three: the same reasoning (data in flight = rate × time) gives the bandwidth–delay product.

TCP (Transmission Control Protocol) · RTT (round-trip time)

Problem 6 · Ten transfers through one shared core link



The core is the bottleneck when **R / 10 < 8 Mbit/s**, i.e. **R < 80 Mbit/s**.

R	min(20, 8, R/10)	who limits
1 Gbit/s	min(20, 8, 100) = **8 Mbit/s**	the client access link
50 Mbit/s	min(20, 8, 5) = **5 Mbit/s**	the shared core link

Key idea For each flow, compute what every link offers it — the smallest wins, and it can move as R changes.

Problem 7 · Three paths, their bandwidth–delay products

(a) campus LAN · 1 Gbit/s · RTT 0.5 ms

BDP = **62.5 KB**

(b) Bucharest–Frankfurt · 1 Gbit/s · RTT 30 ms



BDP = **3.75 MB**

(c) geostationary satellite · 50 Mbit/s · RTT 600 ms



BDP = **3.75 MB**

(b) and (c): **same area**
a slow satellite link holds as much in flight as a fast terrestrial one

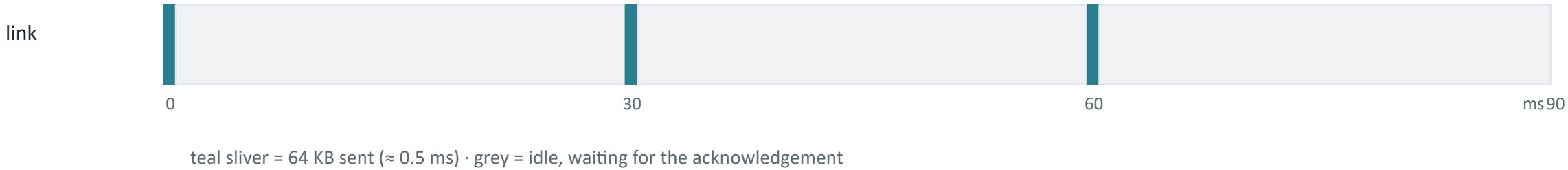
drawn to scale; (a) is a sliver:
10^9 bit/s × 0.0005 s = 0.5 Mbit = 62.5 KB

Key idea BDP = rate × RTT: a long path and a fast path can need the same amount of data in flight.

RTT (round-trip time) · LAN (local area network)

Problem 7 · Q2–Q3: a 64 KB window on path (b)

three round trips, to scale (RTT 30 ms)



teal sliver = 64 KB sent (≈ 0.5 ms) · grey = idle, waiting for the acknowledgement

65 535 × 8 bits / 0.03 s ≈ **17.5 Mbit/s**
under 2 % of the 1 Gbit/s link

to fill the path: window ≥ **one BDP = 3.75 MB**

1 The original TCP header could express at most 64 KB; the **window scale** option (RFC 7323) was added to remove this limit.

Key idea A window smaller than the BDP leaves the link idle most of the time, however fast it is.

TCP (Transmission Control Protocol) · RTT (round-trip time) · BDP (bandwidth–delay product) · RFC (Request for Comments)

Problem 8 · Four proposals judged by the end-to-end argument

Routers verify every TCP checksum and drop corrupted segments, so hosts need not check.	✗	**inconsistent as a replacement** corruption also happens inside routers and hosts; only an end-to-end check catches it — and routers do not check TCP checksums in practice, because of the cost
A mobile operator's radio link retransmits lost frames locally a few times.	✓	**consistent: a performance enhancement** TCP still checks end to end; the link only recovers frequent radio losses faster
An ISP's proxy recompresses all images customers download.	!	**problematic** the network changes data the endpoints did not ask to change; a medical image cannot tolerate it — HTTPS now defeats such proxies
Banks encrypt end to end with TLS although the internal network is encrypted link by link.	✓	**consistent** only the endpoints can guarantee confidentiality between them; link encryption leaves data clear inside each device

Key idea Ask of every function: can the network alone get it right? If not, the endpoints must do it — the network may only speed it up.

HTTPS (HyperText Transfer Protocol Secure) · TCP (Transmission Control Protocol) · TLS (Transport Layer Security) · ISP (Internet service provider)

Homework

1 Exercises 1–5

at the end of Lecture 2 — solve them without looking at the answers

2 Decode your own SYN

the first 54 bytes of a TCP SYN (synchronize) segment you capture in Laboratory 2, by hand, like Problem 2; check every field against Wireshark

3 Read and reflect

section 2 of Saltzer–Reed–Clark (careful file transfer); five lines on whether the argument applies to e-mail delivery

Key idea Decoding a frame you captured yourself is the best test that the field map is in your head.

TCP (Transmission Control Protocol)

Part C · Laboratory 2 on Linux

A · Lecture 2
Architecture and performance

→

B · Seminar 2
Headers, queues, throughput

→

C · Laboratory 2 · Linux
Wireshark and tcpdump

→

D · Laboratory 2 · Windows
Wireshark and pktmon

In this part

Before you start · plan

1 · A first capture: the three panes

2 · Capture filters and display filters

3 · One web request, layer by layer

4 · The whole conversation

5 · Measuring RTT from the handshake

6 · Traceroute under the microscope

7 · Command-line capture: tcpdump, tshark

8 · HTTPS: what remains visible

If something does not work

HTTPS (HyperText Transfer Protocol Secure) · RTT (round-trip time)

Before you start (Linux)

1 · install

```
sudo apt install -y wireshark tshark
  tcpdump traceroute curl dnsutils
```

→

2 · allow capture

```
answer “Yes” for non-superusers ·
 sudo usermod -aG wireshark $USER
```

→

3 · log out and in

```
the new group applies at the next
              login
```

→

4 · check

```
groups | grep wireshark
```

Environment
course VM (virtual machine) or Linux with a desktop · in WSL 2 (Windows Subsystem for Linux): tcpdump there, open the files in Wireshark on Windows

Interface name
`ip -br link` → enp0s3, ens33, wlp2s0 … · below, **eth0** stands for yours

Rules of conduct
only your own machine, only your own traffic — capturing others’ is unethical and often illegal

Hand in
answers to **Q1–Q16** + the capture **web.pcapng** from Task 3 · values shown here are illustrations

Key idea Capturing needs permission twice: from the system (the wireshark group) and from the people whose traffic it is.

Laboratory 2 (Linux) · the two hours



1 A first capture and a tour of Wireshark

2 Capture filters and display filters

3 One web request, layer by layer

4 The whole conversation: streams, statistics, flow graph

5 Measuring RTT (round-trip time) from the handshake

6 Traceroute under the microscope

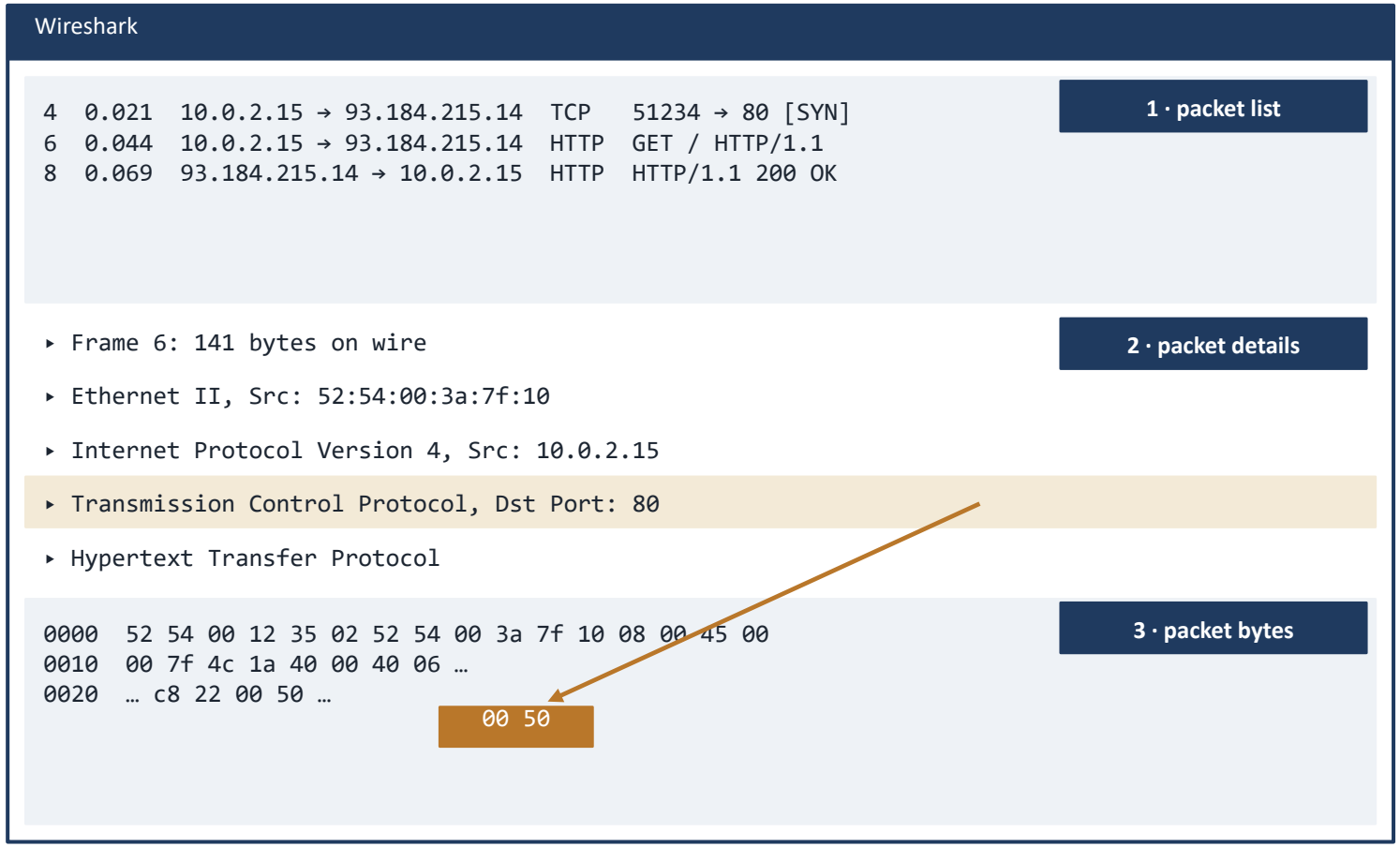
7 Command-line capture: tcpdump and tshark

8 HTTPS: what remains visible

Key idea Task 3 is the heart of the lab: one web request opened at every layer.

HTTPS (HyperText Transfer Protocol Secure)

Task 1 · Wireshark's three panes




```
curl -s http://example.org/ > /dev/null
```

← (capture info)

← link

← network

← transport

← application

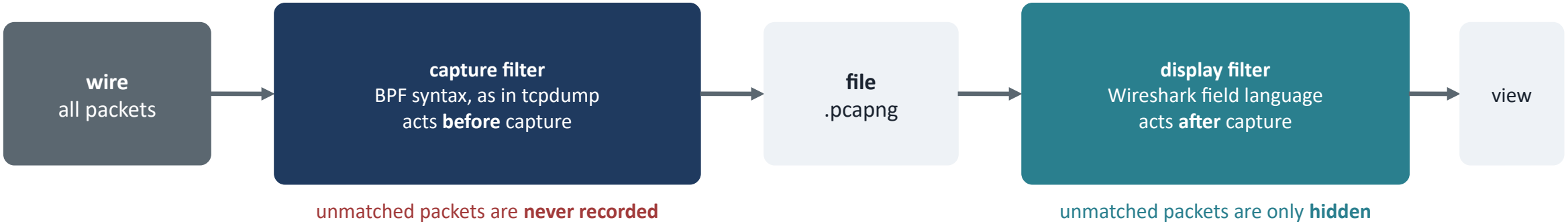
Q1 · how many packets, how many from curl? what else does your machine send?

Q2 · match the detail lines to the five layers

Key idea List = one line per packet; details = one line per layer; bytes = the raw frame. Click to connect them.

SYN (synchronize)

Task 2 · Capture filters vs display filters




```
host example.org · port 53
tcp port 80 · icmp
port 53 or port 80 ← use this one
```

```
dns · http · tcp.port == 80 · icmp
ip.addr == 10.20.0.53
tcp.flags.syn == 1
```

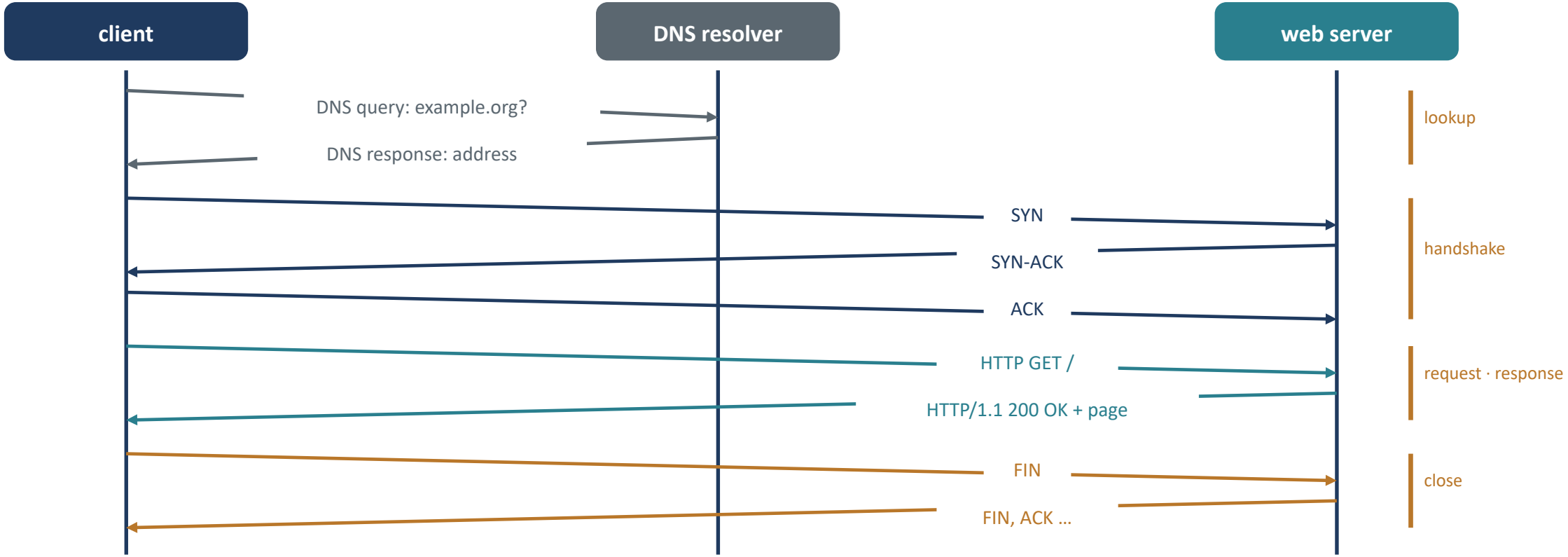
1 **Q3** · display filters for: only DNS responses (hint: `dns.flags.response`) · only packets sent **by** your machine.

Key idea Same idea, two languages: BPF (Berkeley Packet Filter) before recording, Wireshark fields after.

DNS (Domain Name System)

Task 3 · One web request, packet by packet

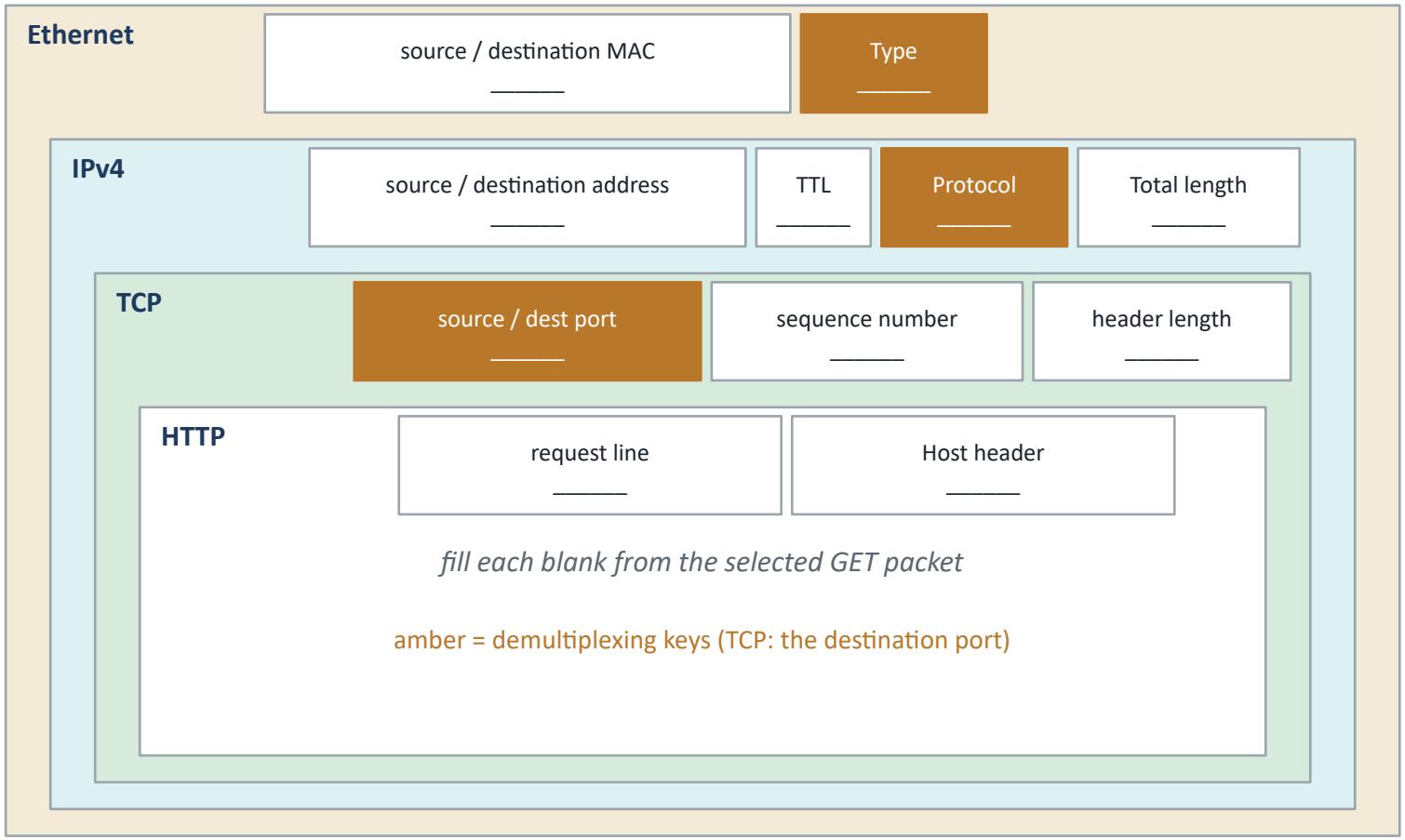
```
resolvectl flush-caches   →   capture   →   curl -s http://example.org/ > /dev/null   →   stop   →   File → Save As… web.pcapng
```



Key idea One small page: lookup, handshake, request, response, close — find them in this order.

HTTP (HyperText Transfer Protocol) · DNS (Domain Name System) · SYN (synchronize) · ACK (acknowledgement) · FIN (finish)

Task 3 · The GET packet, layer inside layer



1 **Q4** · the amber fields are the demultiplexing keys. With EtherType 0x86DD the payload would go to **IPv6**.

2 **Q7** · headers before the first HTTP byte: 14 + 20 + 20 = **54** bytes (Lecture 2, § 3.1). Often you see **66**: Linux adds 12 bytes of TCP options (timestamps).

Key idea Every field the lecture drew is in this one packet — the amber ones are how it reached curl.

HTTP (HyperText Transfer Protocol) · IPv4 (Internet Protocol version 4) · IPv6 (Internet Protocol version 6) · TCP (Transmission Control Protocol) · MAC (Media Access Control) · TTL (time to live)

Task 3 · Q5–Q6: whose MAC, how many routers?




```
GET: dst MAC = 52:54:00:12:35:02
```

```
$ ip neigh → 10.0.2.2 lladdr 52:54:00:12:35:02
```

Q5 · same address: the frame goes to the next hop only.

Q6 · counting routers from the reply's TTL

64	128	255
Linux, most Unix	Windows	many routers

reply TTL 52 → next initial value above is 64 → 64 − 52 = **12 routers**

1 Forward and return paths may differ in length: routing is chosen separately in each direction, by different networks.

Key idea The MAC address you see is your gateway's; the TTL you receive is an initial value minus the routers on the way back.

MAC (Media Access Control) · TTL (time to live) · PC (personal computer)

Task 4 · Four views of one conversation

Follow → TCP Stream

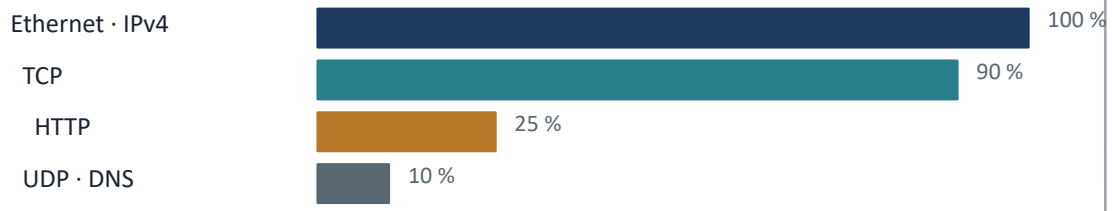
application data of both directions, in colour

```
GET / HTTP/1.1  ·  Host: example.org
```

```
HTTP/1.1 200 OK  ·  Content-Type: text/html
```

Statistics → Protocol Hierarchy

how the bytes divide between protocols



Statistics → Conversations

every pair of endpoints, bytes exchanged

address A ↔ address B	packets	bytes
10.0.2.15 ↔ resolver	2	≈ 160
10.0.2.15 ↔ server	≈ 10	≈ 2 300

Statistics → Flow Graph

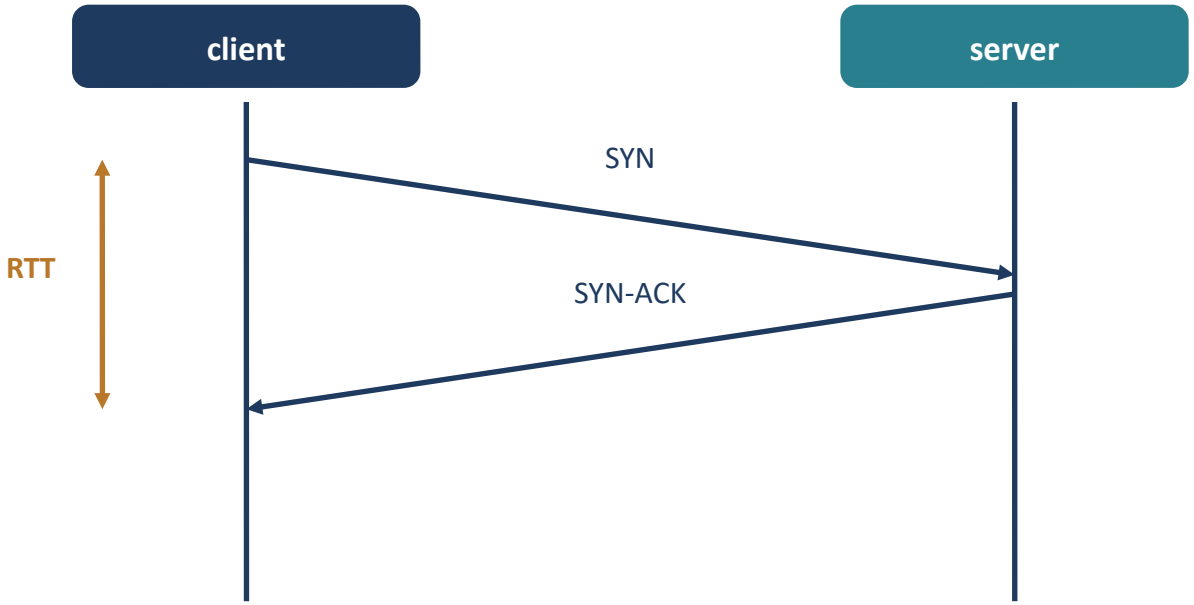
the exchange as a time–sequence diagram



Key idea Q8 · Paste the flow graph and label lookup, handshake, request, response, close — then count the packets.

HTTP (HyperText Transfer Protocol) · IPv4 (Internet Protocol version 4) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · DNS (Domain Name System) · SYN (synchronize) · ACK (acknowledgement)

Task 5 · Measuring RTT from the handshake



No.	Time	Info
4	0.000	51234 → 80 [SYN]
5	0.031	80 → 51234 [SYN, ACK]

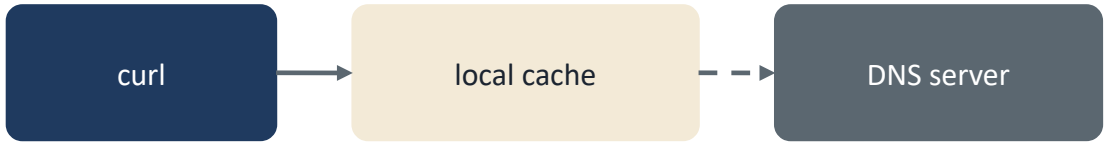
Q9 · compare three numbers

SYN → SYN-ACK time · `ping -c 5 example.org`

the field `tcp.analysis.ack_rtt` (SEQ/ACK analysis)

differences: different packets, server and routers handle them differently

Q10 · the DNS lookup, then the cache

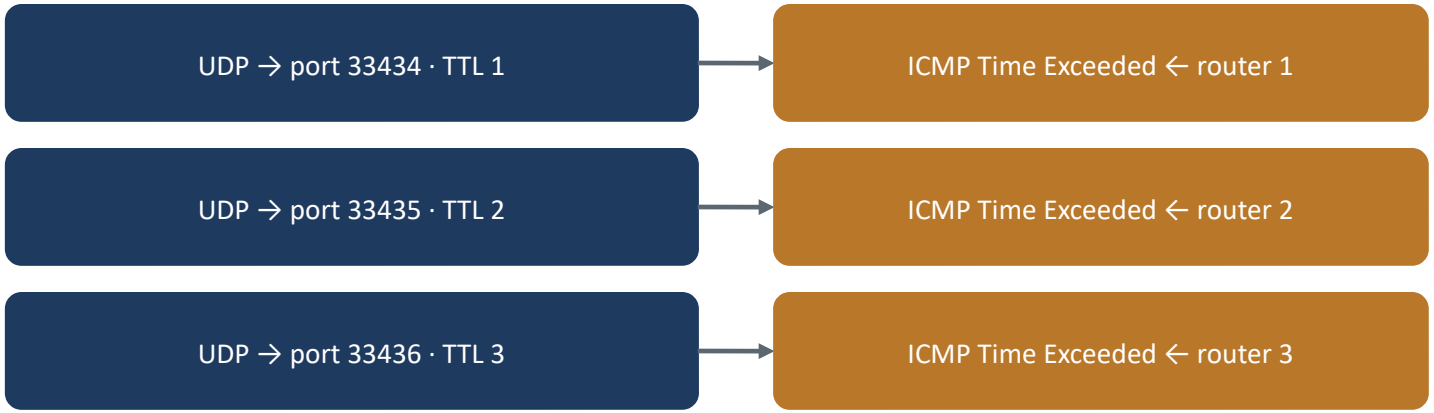


1st curl: miss → a real query, time it. 2nd curl: **hit** → no DNS packets, until the record's TTL expires.

Key idea The handshake is a free RTT measurement inside every TCP connection.

TCP (Transmission Control Protocol) · DNS (Domain Name System) · RTT (round-trip time) · TTL (time to live) · SYN (synchronize) · ACK (acknowledgement) · SEQ (sequence)

Task 6 · Traceroute in the capture



Q11 · protocol, destination ports and TTL of the first three probes

Q12 · inside one Time Exceeded

IP header: router → you

ICMP type 11, code 0

copy of **your** probe: its IP header + first 8 bytes of UDP (the ports)

Why the copy matters

Several probes are in flight at once. The quoted header and ports tell traceroute **which** probe expired, and so which hop answered.

Key idea Each reply carries the start of the probe that caused it — that is how traceroute matches answers to questions.

ICMP (Internet Control Message Protocol) · UDP (User Datagram Protocol) · TTL (time to live) · IP (Internet Protocol)

Task 7 · Command-line capture: tcpdump and tshark

capture to the screen	`sudo tcpdump -i eth0 -n -c 20 'port 53 or port 80'`	→ 20 packets printed, one per line
capture to a file	`sudo tcpdump -i eth0 -n -w cli.pcap 'host example.org' & then curl … ; sudo pkill tcpdump`	→ cli.pcap
read the file	`tshark -r cli.pcap`	→ one summary line per packet
extract fields	`tshark -r cli.pcap -Y http.request -T fields -e ip.src -e ip.dst -e http.host -e http.request.uri`	→ columns: source, destination, host, path

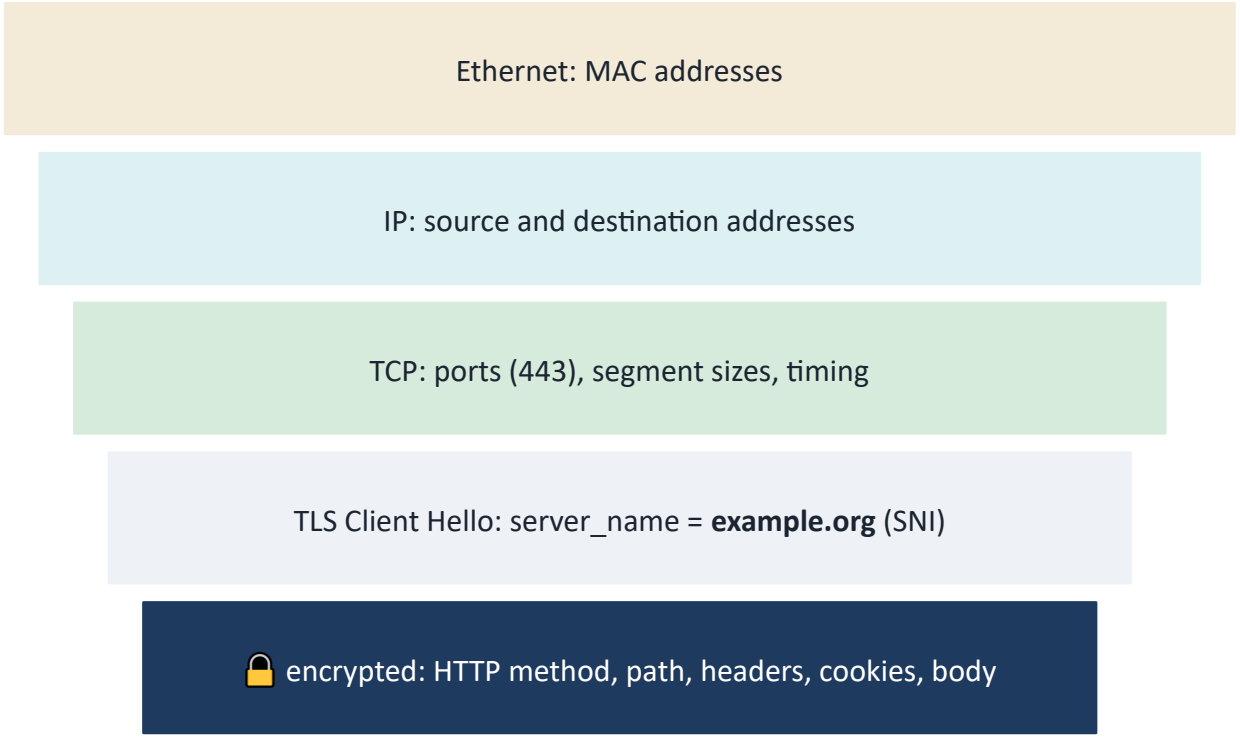
1 **-n**: numbers, not names — no reverse DNS lookups (which would add packets of their own).

2 **Q13** · open cli.pcap in Wireshark: is it the same conversation as in Task 3?

Key idea tcpdump uses capture-filter (BPF, Berkeley Packet Filter) syntax; tshark -Y uses display-filter syntax.

DNS (Domain Name System)

Task 8 · HTTPS: what remains visible



top four: **in clear** · bottom: **encrypted** by TLS (Transport Layer Security)

Q15 · an observer still learns

- your and the server's addresses
- the name: SNI (Server Name Indication) and the earlier DNS query
- sizes, times, number of connections

Q16 · encrypted: the application layer

headers below travel in clear; only the two endpoints hold the keys → the end-to-end argument

Key idea HTTPS hides what you ask for, not whom you ask, when, or how much.

HTTPS (HyperText Transfer Protocol Secure) · HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol) · DNS (Domain Name System) · MAC (Media Access Control) · IP (Internet Protocol)

If something does not work (Linux)

Wireshark lists no interfaces	not yet in the `wireshark` group: run `groups`; log out and in again
no DNS query appears	the answer was cached: `resolvectl flush-caches`, or query another name
no HTTP, only TLS	redirected to HTTPS: use `http://example.org/` exactly, `curl` without `-L`
capture flooded with other traffic	use a capture filter, or a display filter on your server's address
`tcpdump: permission denied`	prefix with `sudo`, or use Wireshark's group permission

Key idea Most problems are permissions, caches or redirects — check those three first.

HTTPS (HyperText Transfer Protocol Secure) · HTTP (HyperText Transfer Protocol) · TLS (Transport Layer Security) · DNS (Domain Name System)

Part D · Laboratory 2 on Windows

A · Lecture 2
Architecture and performance

→

B · Seminar 2
Headers, queues, throughput

→

C · Laboratory 2 · Linux
Wireshark and tcpdump

→

D · Laboratory 2 · Windows
Wireshark and pktmon

In this part

Before you start: how Windows captures

3 · One web request, layer by layer

6 · tracert under the microscope

8 · HTTPS: what remains visible

1–2 · First capture, filters

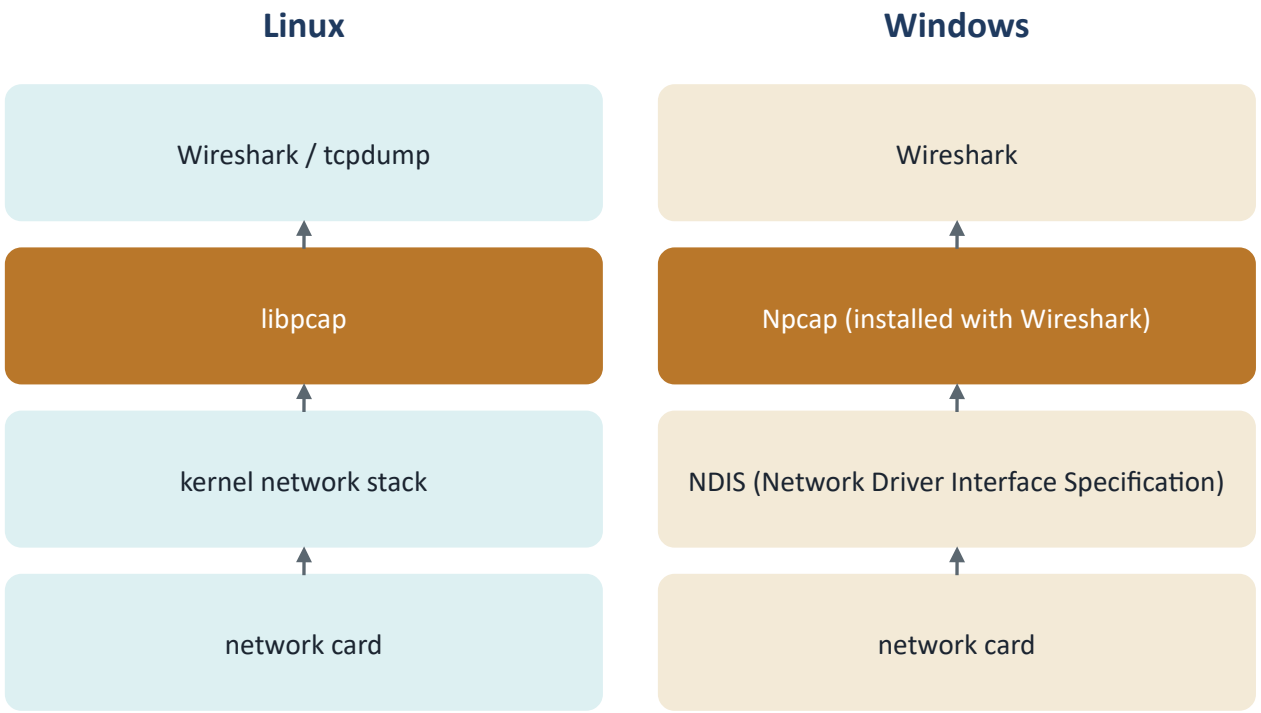
4–5 · Conversation views and RTT

7 · pktmon and tshark

Linux and Windows side by side

HTTPS (HyperText Transfer Protocol Secure) · RTT (round-trip time)

Before you start (Windows): how packets reach Wireshark



Npcap = the Nmap project's packet capture driver · libpcap = the packet capture library

Install
Wireshark from wireshark.org; accept **Npcap**. Installing needs administrator rights; capturing afterwards does not.

Terminal
PowerShell; commands marked *(admin)* need "Run as administrator".

Wi-Fi note
Windows hands Wireshark frames that **look like** Ethernet; real 802.11 headers only in monitor mode (same on Linux).

Hand in
answers to **Q1–Q16** + **web.pcapng** from Task 3; values shown are illustrations.

Key idea Same Wireshark, different plumbing: Npcap plays on Windows the role libpcap plays on Linux.

Laboratory 2 (Windows) · the two hours



1 A first capture and a tour of Wireshark

2 Capture filters and display filters

3 One web request, layer by layer

4 The whole conversation: streams, statistics, flow graph

5 Measuring RTT (round-trip time) from the handshake

6 tracert under the microscope

7 Command-line capture: pktmon and tshark

8 HTTPS: what remains visible

Key idea Identical structure to the Linux sheet, so the answers can be compared task by task.

HTTPS (HyperText Transfer Protocol Secure)

Tasks 1–2 · First capture on Windows

Ethernet



Wi-Fi

VPN adapter

vEthernet (Hyper-V)

← double-click the one that moves

```
curl.exe -s http://example.org/ -o NUL
```

Q1 · other protocols you will see without asking:

ARP
Address Resolution Protocol

mDNS
multicast DNS (Domain Name System)

SSDP
Simple Service Discovery Protocol

LLMNR
Link-Local Multicast Name Resolution

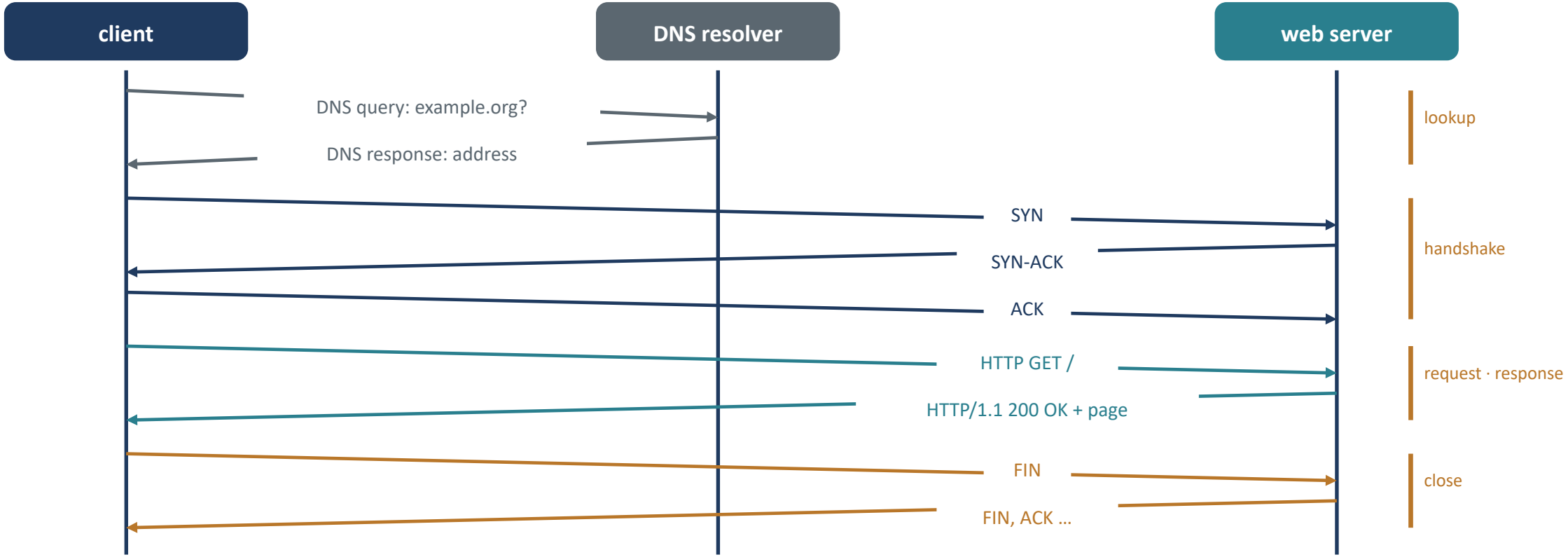
Q2–Q3 · match the detail lines to the five layers · the filters are exactly those of the Linux sheet:
capture `port 53 or port 80` · display `dns`, `http`, `tcp.port == 80`, `tcp.flags.syn == 1` · your own address: `ipconfig`

Key idea On Windows choose the adapter by its traffic, not by its name: virtual adapters show nothing.

mDNS (multicast DNS, Domain Name System) · VPN (virtual private network)

Task 3 · One web request on Windows

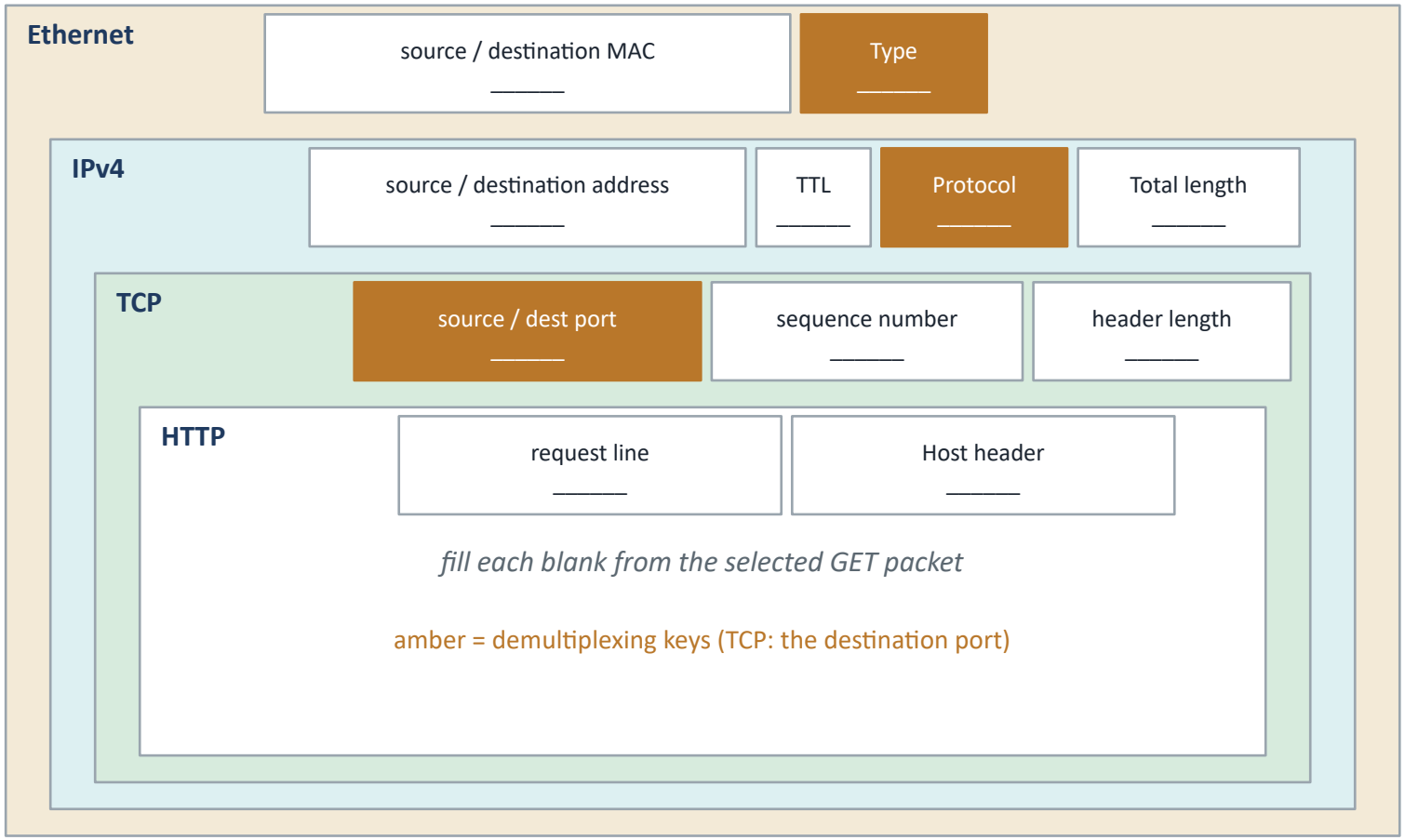
```
ipconfig /flushdns   →   capture   →   curl.exe -s http://example.org/ -o NUL   →   stop   →   File → Save As… web.pcapng
```



Key idea The packets are the same as on Linux — only the commands that produce them differ.

HTTP (HyperText Transfer Protocol) · DNS (Domain Name System) · SYN (synchronize) · ACK (acknowledgement) · FIN (finish)

Task 3 · The GET packet and Q5–Q7 on Windows



1 **Q5** · compare the GET's destination MAC with your gateway's: `arp -a` or `Get-NetNeighbor` — they match.

2 **Q6** · your outgoing TTL: **128** (Windows). Reply TTL 52 → from 64 → 12 routers. **Q7** · 14 + 20 + 20 = 54 bytes of headers before HTTP.

Key idea Same demultiplexing keys, same headers; Windows differs only in its initial TTL and its tools.

HTTP (HyperText Transfer Protocol) · IPv4 (Internet Protocol version 4) · TCP (Transmission Control Protocol) · MAC (Media Access Control) · TTL (time to live)

Tasks 4–5 · Conversation views and RTT on Windows

Follow → TCP Stream

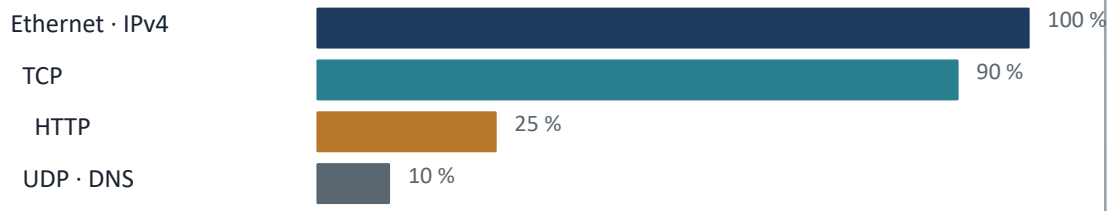
application data of both directions, in colour

```
GET / HTTP/1.1  ·  Host: example.org
```

```
HTTP/1.1 200 OK  ·  Content-Type: text/html
```

Statistics → Protocol Hierarchy

how the bytes divide between protocols



Statistics → Conversations

every pair of endpoints, bytes exchanged

address A ↔ address B	packets	bytes
10.0.2.15 ↔ resolver	2	≈ 160
10.0.2.15 ↔ server	≈ 10	≈ 2 300

Statistics → Flow Graph

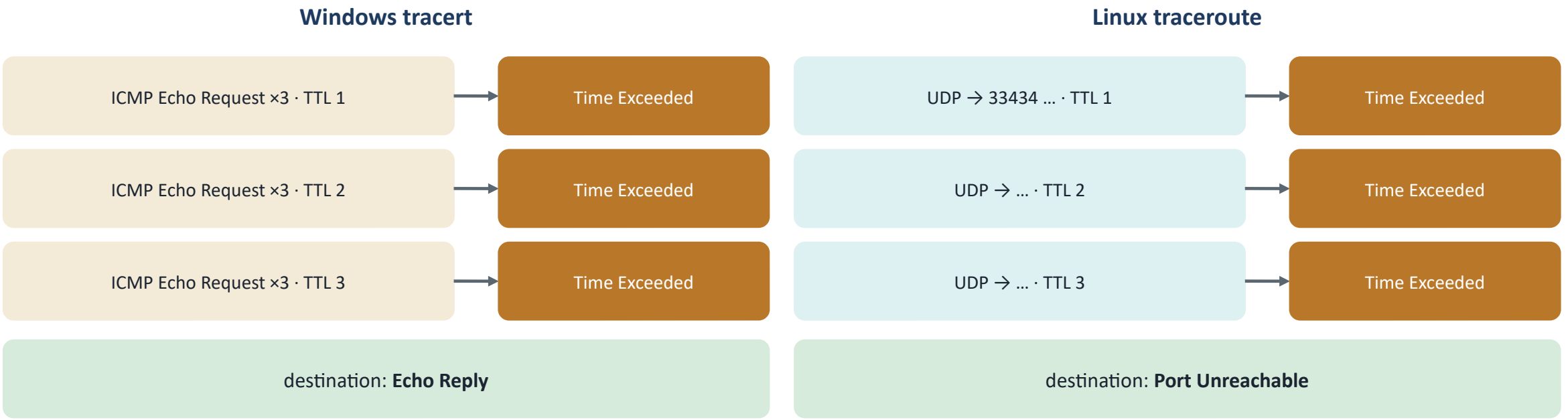
the exchange as a time–sequence diagram



Key idea Q9–Q10 · ping -n 5 replaces ping -c 5; ipconfig /displaydns shows the cached name and its remaining TTL.

HTTP (HyperText Transfer Protocol) · IPv4 (Internet Protocol version 4) · TCP (Transmission Control Protocol) · UDP (User Datagram Protocol) · DNS (Domain Name System) · RTT (round-trip time) · TTL (time to live) · SYN (synchronize) · ACK (acknowledgement)

Task 6 · tracert sends ICMP, not UDP



1 **Q11–Q12** · probe type, TTL hop after hop, probes per hop; replies are ICMP Time Exceeded (type 11) and quote the start of **your** Echo Request — so tracert knows which probe expired.

Key idea Same TTL trick, different probe: ICMP Echo on Windows, UDP to high ports on Linux.

ICMP (Internet Control Message Protocol) · UDP (User Datagram Protocol) · TTL (time to live)

Task 7 · pktmon: Windows' built-in packet monitor

(admin) start

```
pktmon start --capture --comp nics --pkt-size 0 -f cli.etl
```

traffic

```
curl.exe -s http://example.org/ -o NUL
```

stop

```
pktmon stop
```

convert

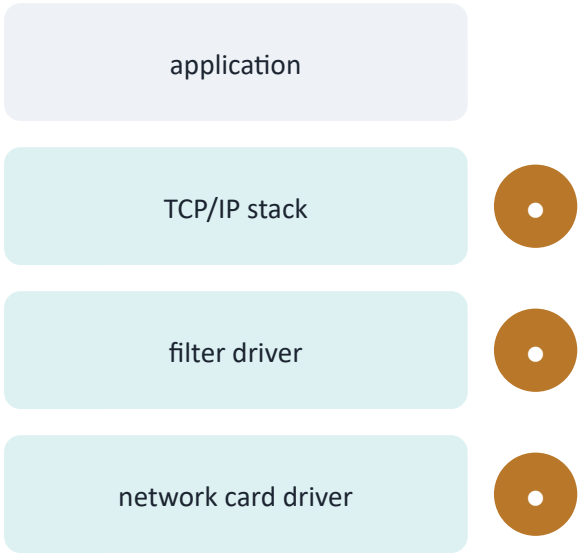
```
pktmon etl2pcap cli.etl -o cli.pcapng
```

analyse

```
tshark.exe -r cli.pcapng -Y http.request -T fields -e ip.src -e ip.dst -e http.host -e
http.request.uri
```

ETL = Event Trace Log · tshark.exe lives in C:\Program Files\Wireshark

Q13 · why duplicates?

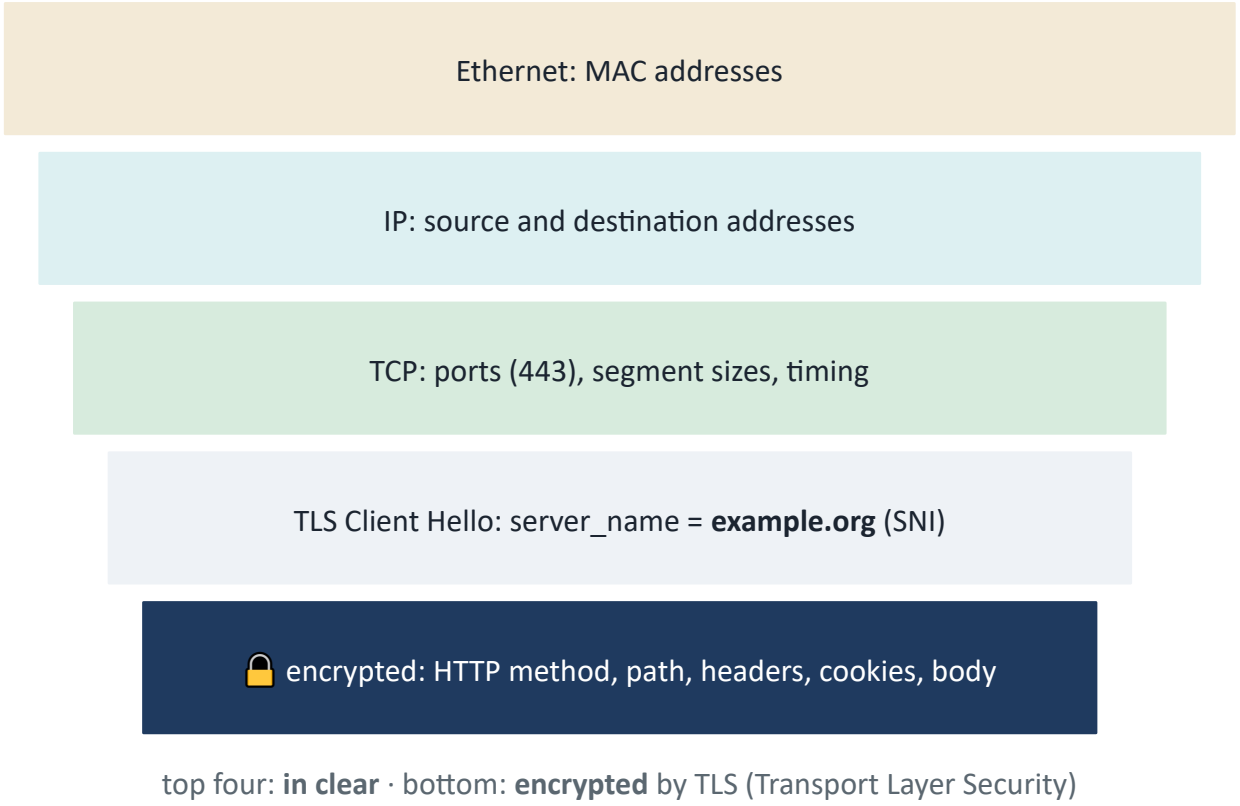


pktmon can record a packet at **several points** in the stack, so it may appear more than once.

Key idea pktmon → .etl → etl2pcap → .pcapng: the built-in way to capture when Wireshark is not installed.

TCP (Transmission Control Protocol) · IP (Internet Protocol)

Task 8 · HTTPS on Windows: what remains visible



top four: **in clear** · bottom: **encrypted** by TLS (Transport Layer Security)

Q15 · an observer still learns

- your and the server's addresses
- the name: SNI (Server Name Indication) and the earlier DNS query
- sizes, times, number of connections

Q16 · encrypted: the application layer

headers below travel in clear; only the two endpoints hold the keys → the end-to-end argument

Key idea Encryption protects the content end to end; addresses, names, sizes and times remain visible on any system.

HTTPS (HyperText Transfer Protocol Secure) · HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol) · DNS (Domain Name System) · MAC (Media Access Control) · IP (Internet Protocol)

Linux and Windows side by side

Purpose	Linux	Windows
capture driver	kernel (libpcap)	Npcap
graphical capture	Wireshark	Wireshark
built-in command-line capture	tcpdump	pktmon (admin)
command-line analysis	tshark	tshark.exe
flush the DNS cache	resolvectl flush-caches	ipconfig /flushdns
traceroute probes	UDP to high ports (default)	ICMP Echo Request
typical initial TTL	64	128

Key idea Learn the purpose once; on each system only the tool's name and a few defaults change.

Npcap (the Nmap project's packet capture driver) · ICMP (Internet Control Message Protocol) · UDP (User Datagram Protocol) · DNS (Domain Name System) · TTL (time to live)

If something does not work (Windows)

Wireshark lists no interfaces	Npcap missing: re-run the Wireshark installer and select Npcap
the chosen adapter shows nothing	a virtual adapter (VPN, virtual private network; Hyper-V): choose the one whose activity line moves
no DNS query appears	the name was cached: `ipconfig /flushdns` again just before the request
`pktmon` reports “access denied”	open the terminal with “Run as administrator”
`curl` prints an object, not a page	that was the PowerShell alias: type `curl.exe`

Key idea On Windows the usual culprits are the driver, the adapter and the curl alias.

Npcap (the Nmap project’s packet capture driver) · DNS (Domain Name System)

Next week

Lecture 3

application architectures · sockets and the services applications need · the Web and HTTP/1.1

Seminar 3

problems on page-load times, messages and framing

Laboratory 3

a TCP client and server using sockets — in C on Linux, with Winsock on Windows

Key idea This week you read packets that others' programs sent; next week you write the programs.

HTTP (HyperText Transfer Protocol) · TCP (Transmission Control Protocol)